

Time Series Analysis

Machine Learning for Time Series

Dr. Ludovic Amruthalingam
ludovic.amruthalingam@hslu.ch

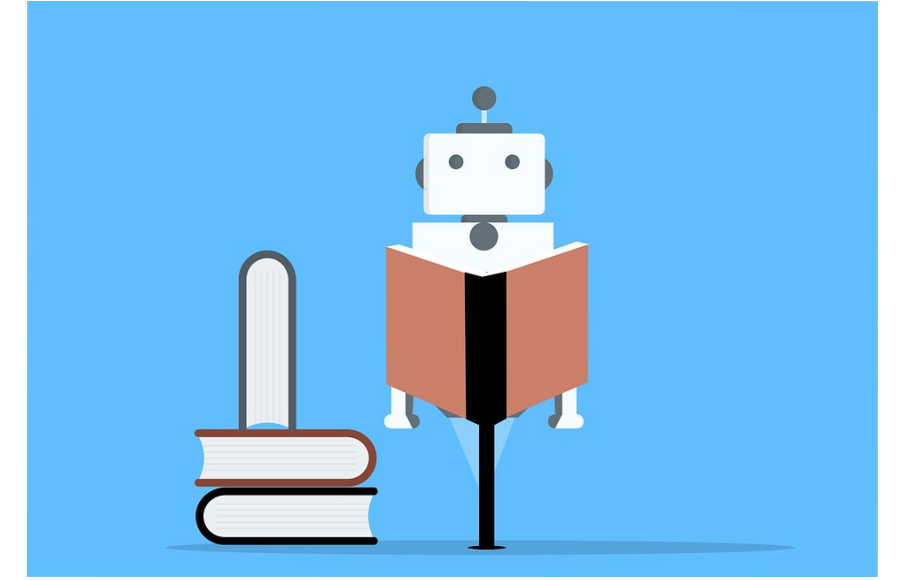
Informatik



Outline

- Machine learning
- Learning tasks
- Feature engineering
- Feature importance and selection
- Tour of ML models
- Evaluation workflow
- Challenges

Machine learning



Machine learning studies **algorithms** that autonomously “learn” to perform **tasks** from **data** X .

$$t : X \rightarrow Y$$

Instead of manually defining specific steps and instructions, these algorithms are designed to automatically **extract patterns and statistical relationships** from data that are useful to complete the task.

$$m_{\theta} : X \rightarrow Y$$

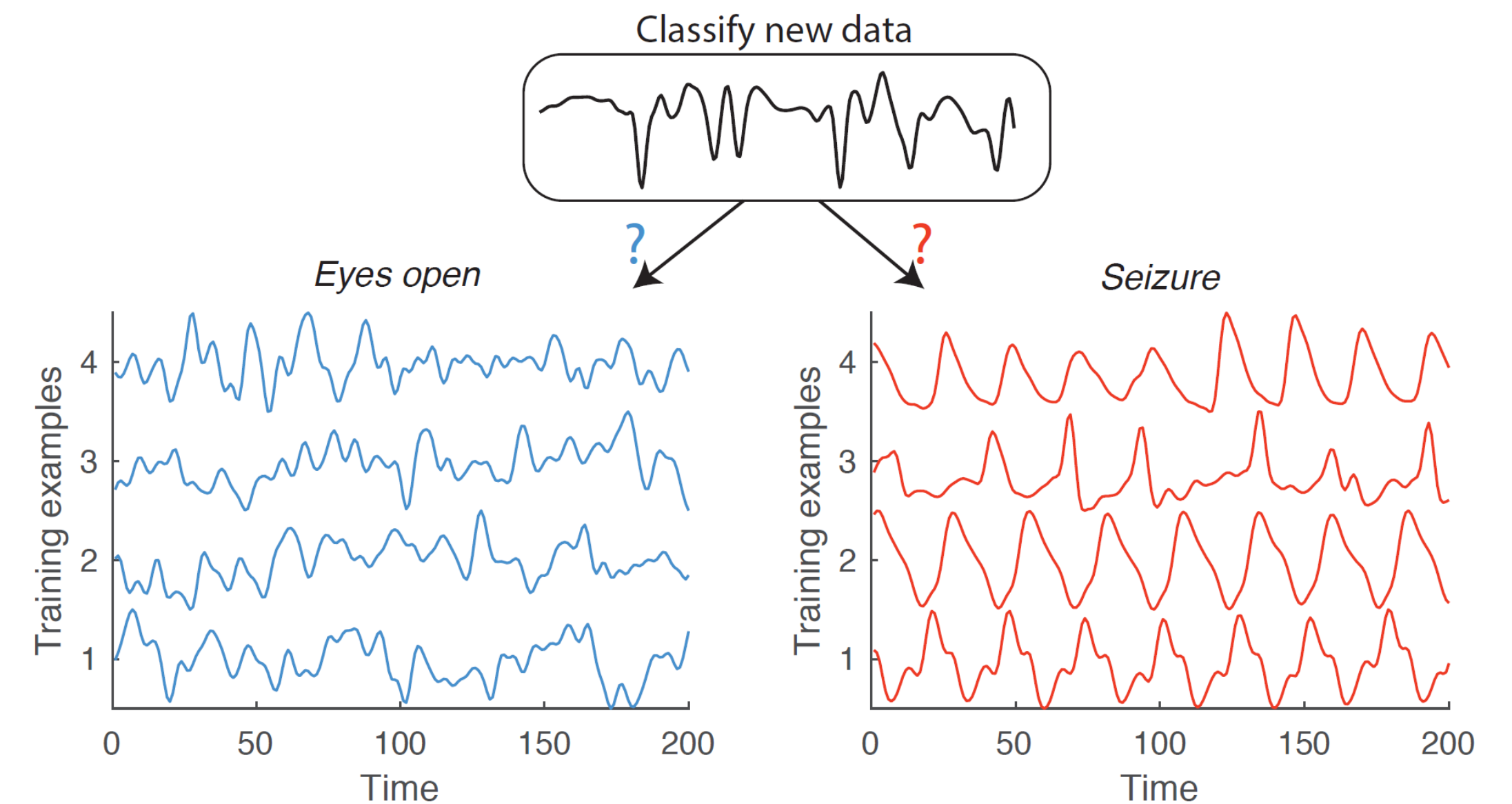
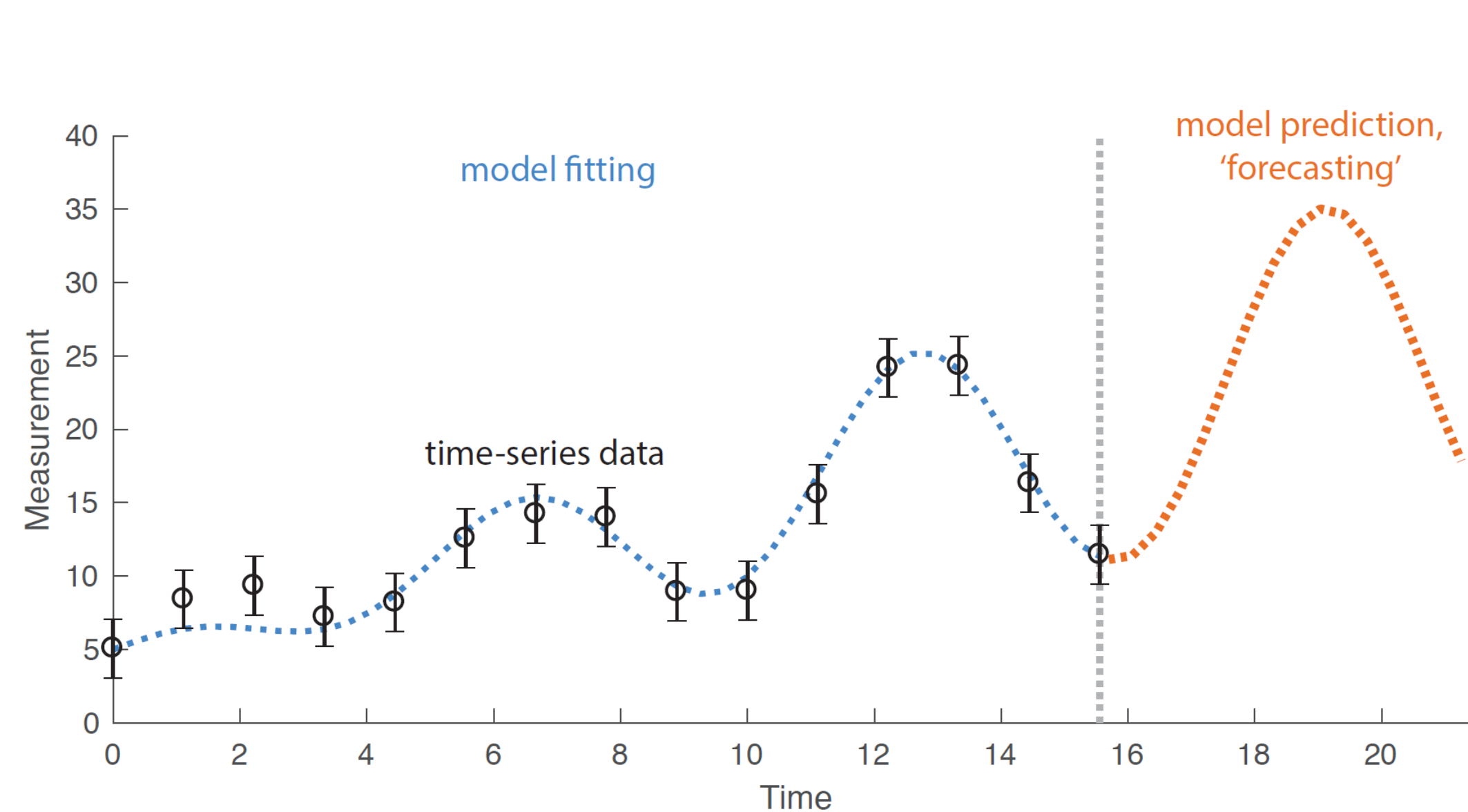
An algorithm is considered to “learn” if its **performance** improves as it processes data. The training procedure optimizes the parameters θ of the algorithm’s model m_{θ} with respect to a performance measure P :

$$\theta^* = \max_{\theta} \sum_{\forall x \in X} P(m_{\theta}(x), t(x))$$

Learning tasks – Supervised learning

Supervised learning: **both** the task **data** X **and target** Y are available.

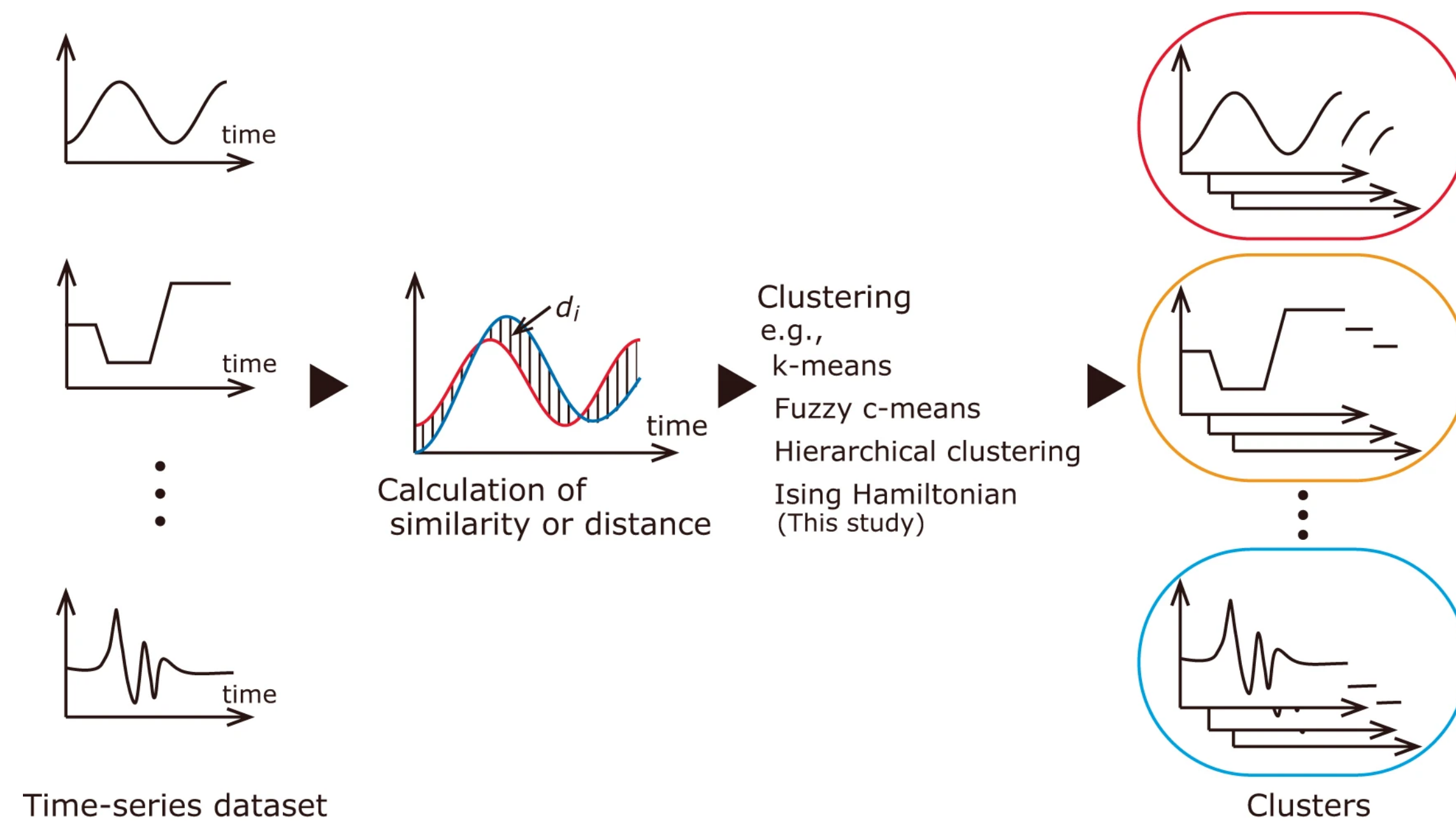
- Regression
- Classification



Learning tasks – Unsupervised learning

Unsupervised learning: **only** the task **data** X is available.

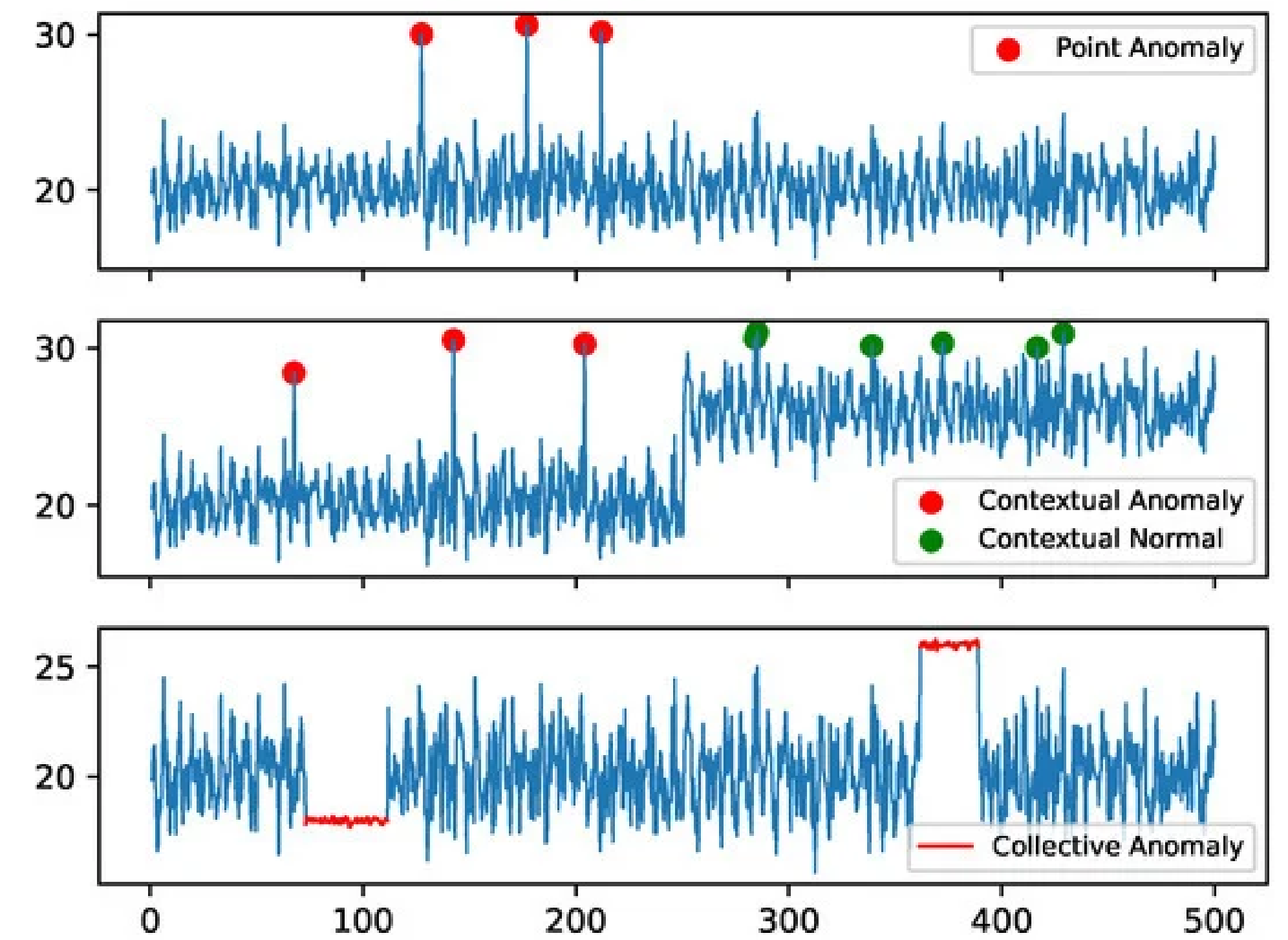
- Clustering
- Anomaly detection



Time-series dataset

Clusters

HSLU Inoue, Tomoki, et al. "Clustering method for time-series images using quantum-inspired digital annealer technology." (2024)



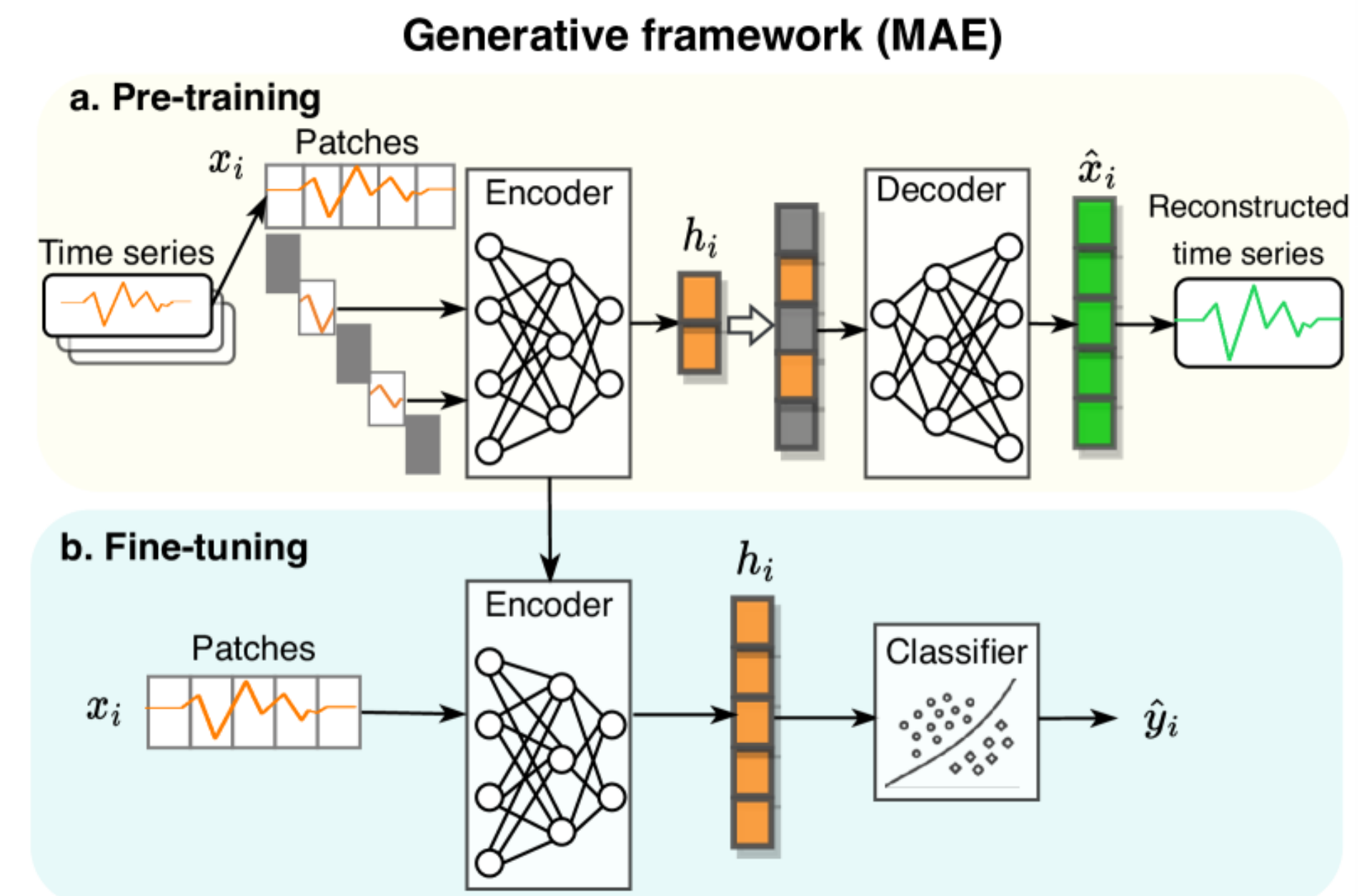
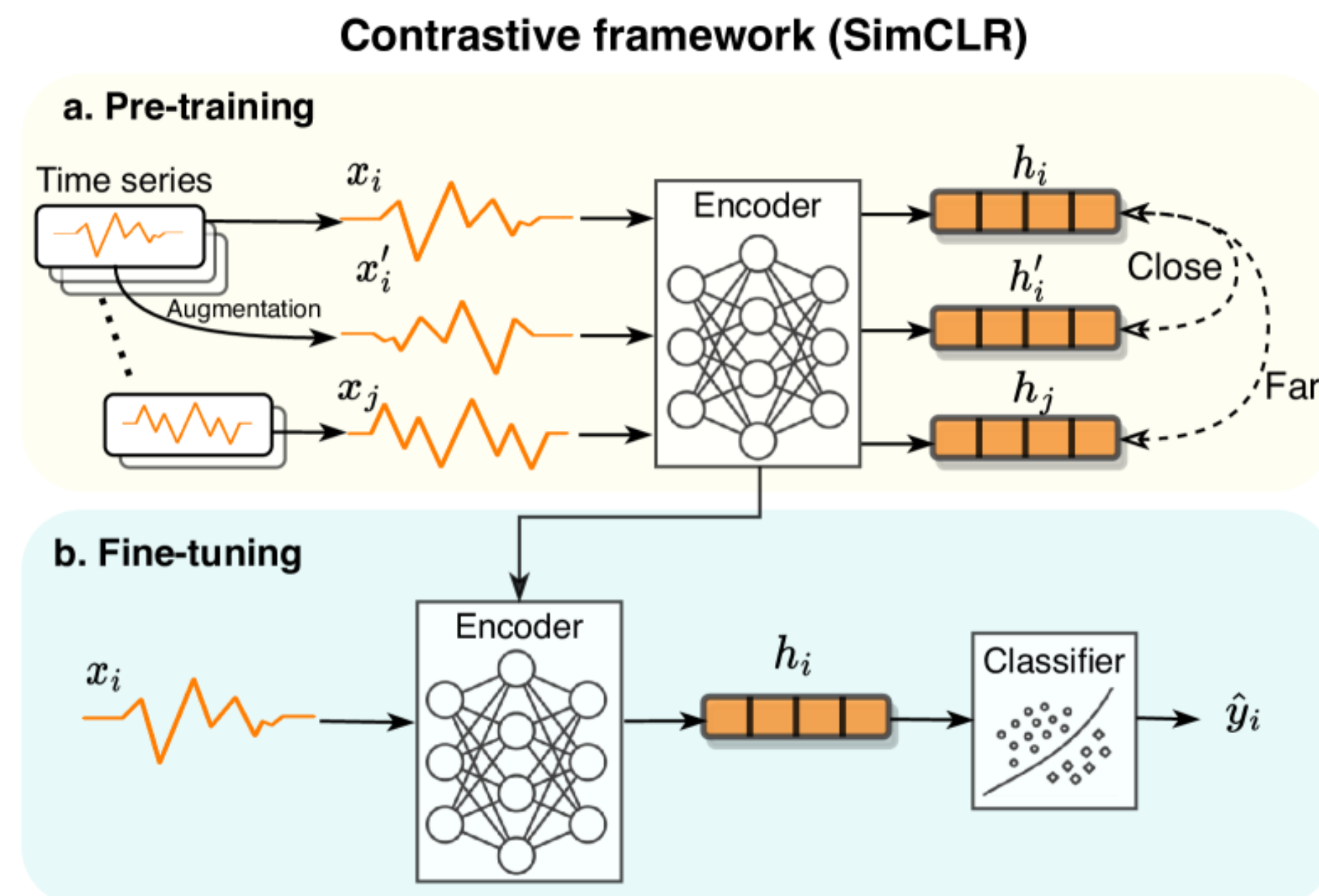
Belay, Mohammed Ayalew, et al. "Unsupervised anomaly detection for IoT-based multivariate time series: Existing solutions, performance analysis and future directions." (2023) Slide 5

Learning tasks – Variants

Semi-supervised learning

Self-supervised learning

Reinforcement learning: optimize agent's strategy to maximize a reward scheme.



From statistical models to machine learning models

Classical time-domain models (ARIMA, SARIMA, ETS) explicitly model **temporal structure**.

- These models rely on assumptions about trend, seasonality, autocorrelation, and error structure.

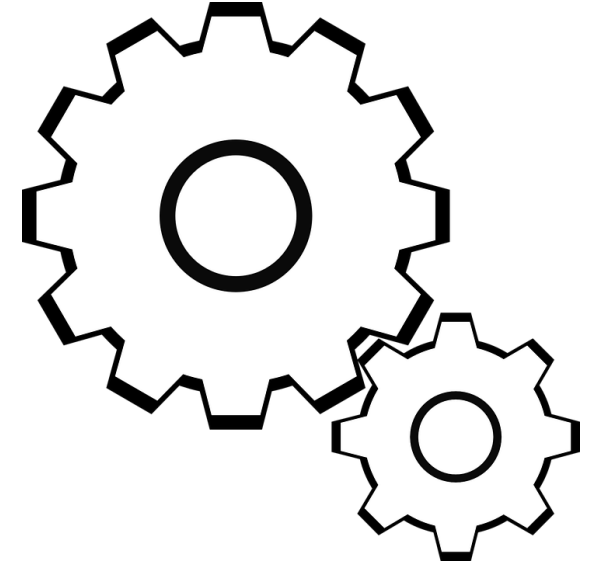
ML algorithms treat time series as **sets of features and targets**.

- They do not model temporal dependencies natively → require **feature engineering**.
- Lag features $\approx$ AR terms
- Rolling statistics $\approx$ moving average components
- Fourier terms $\approx$ seasonal components.

ML generalizes beyond linearity but discards the explicit stochastic structure of TS models.

- More flexibility but **loose interpretability** and the stochastic foundations enabling **uncertainty estimates**.
- **Generalization** of time-domain methods.

Feature engineering for time series



ML models typically assume the samples are **independent and identically distributed (iid)**.

Time series data violates this assumption due to the **correlation between consecutive observations**.

- Each data point depends on previous time steps, creating temporal dependencies.

ML models do not directly incorporate **temporal dependencies** limiting their ability to fit time series patterns.

It is necessary to engineer features **capturing the temporal aspect** of the data

- Time: identify when events occurred e.g., weekday, month, year.
- Passage of time: elapsed time since a reference point.
- Periodicity of time: seasonality, cyclical encoding.

Time series modeling assumptions

Modeling time series requires to respect **order** and **dependency structure**.

Markov assumption (order p): future depends on a **limited history**.

$$P(X_t | X_{1:t-1}) \approx P(X_t | X_{t-p:t-1})$$

This assumption is built-in classical TS models e.g. AR(p).

With ML models, we must **manually** choose lag window size, forecasting horizon and multi-output strategy.

These choices define how the ML model “sees” the TS:

- Too few lags prevents the model from capturing long-term patterns.
- Too many create high-dimensional inputs prone to overfitting.
- Multi-step forecasting greatly amplifies these issues, especially when generating predictions recursively.

Lag features

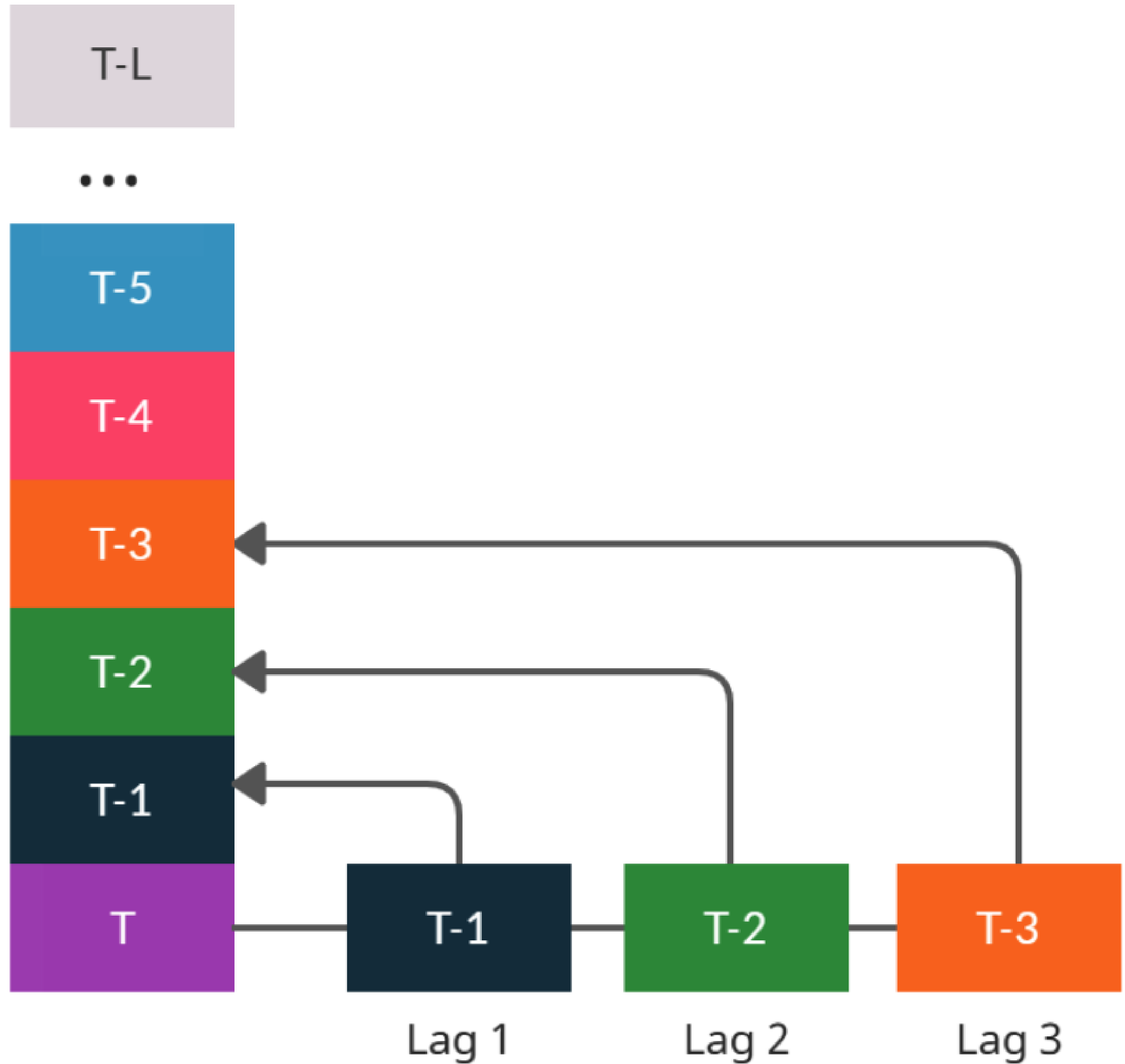
Use previous observations as features.

- Review ACF plot to identify the lags of interest.

Connect **present** observation with **past** observations.

pandas.DataFrame.shift

Current Timestamp



Rolling window aggregation

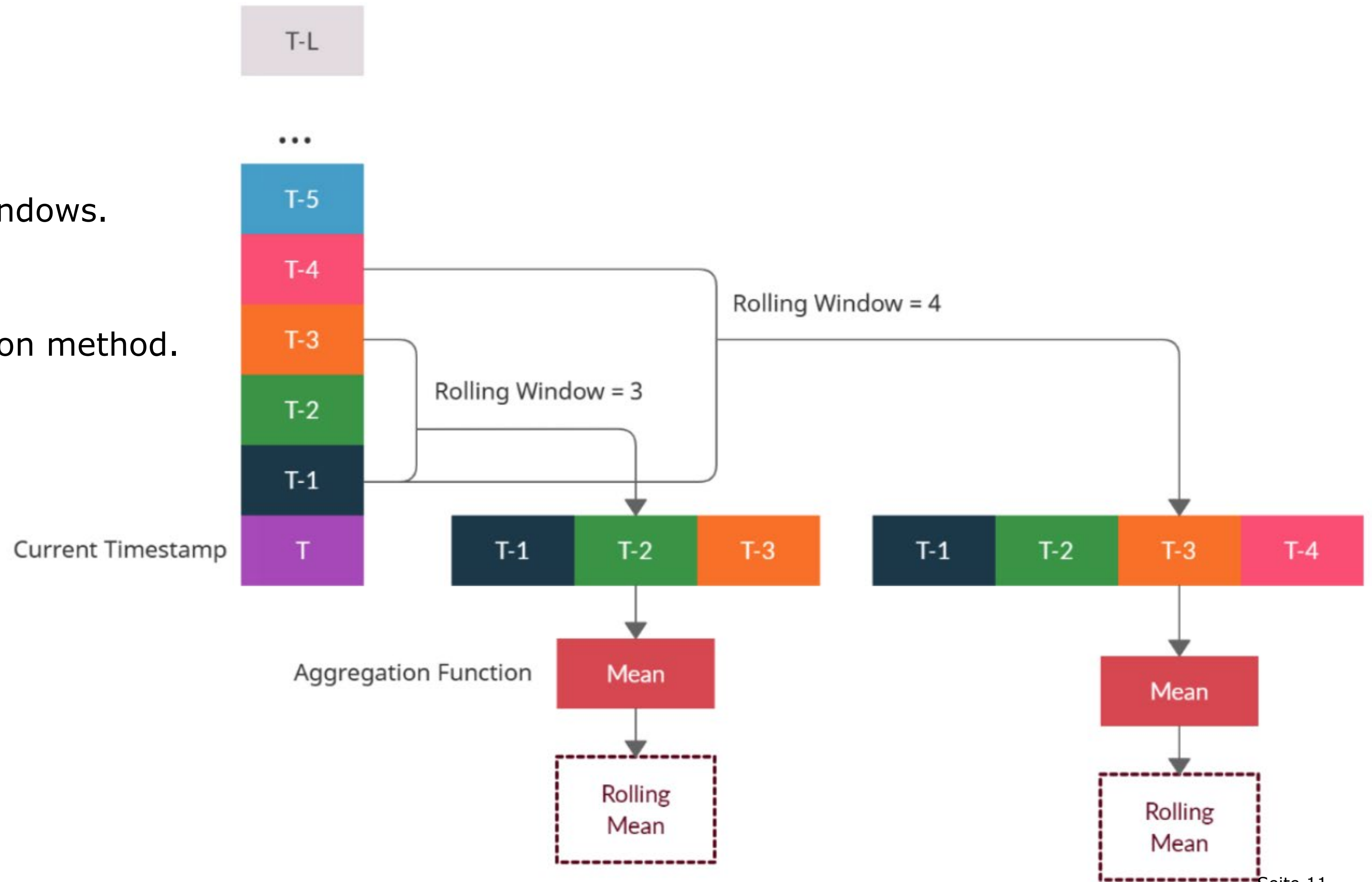
Rolling statistics with different windows.

Consider any **relevant** aggregation method.

- Mean, std, max, min, ...

Do **not** include x_t in the window!

→ data **leakage**



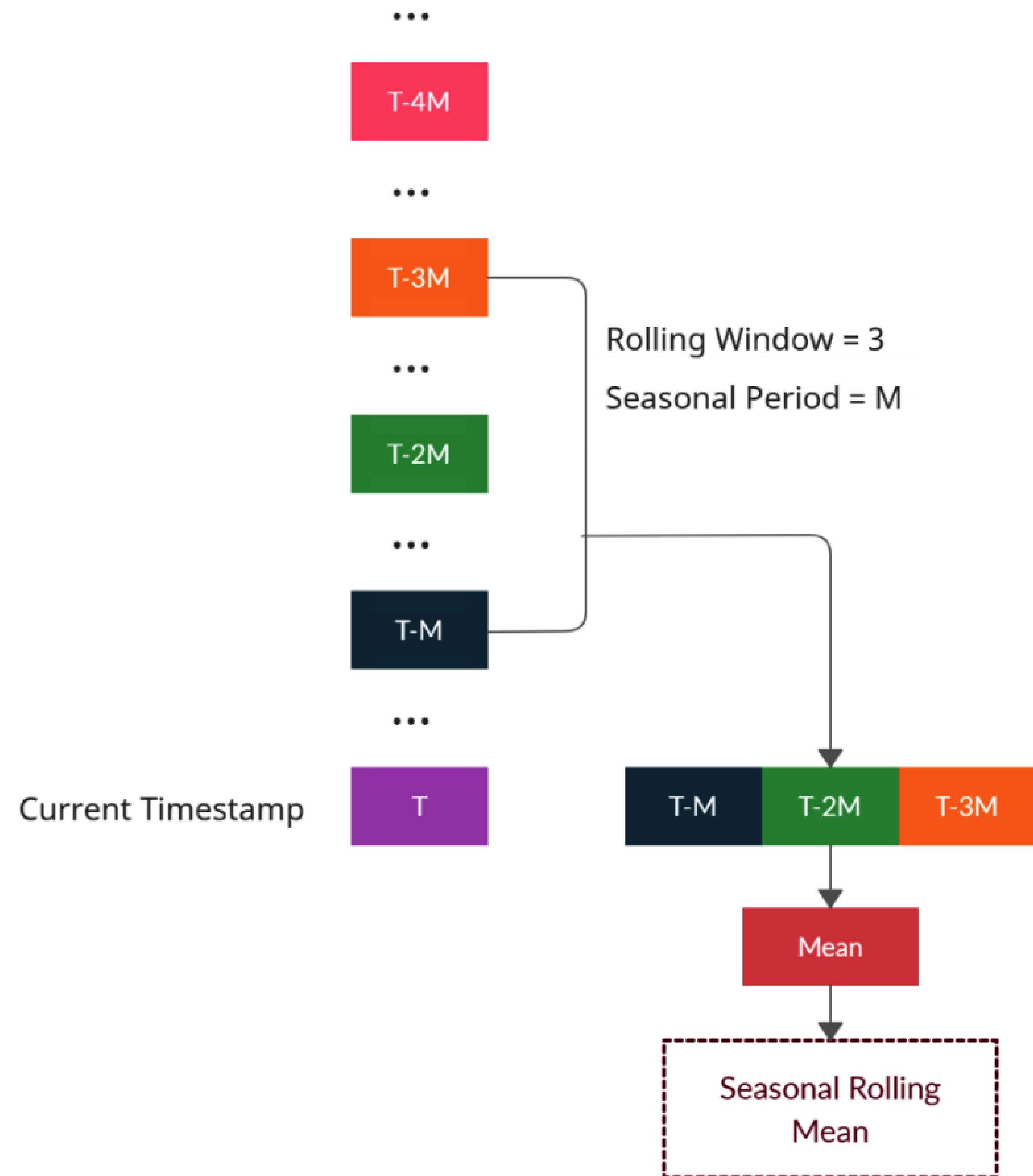
`pandas.DataFrame.rolling`

Seasonal rolling window aggregation

Consider windows with **seasonal observations**.

(Do **not** include x_t in the window! → data **leakage**)

Check the [window-ops](#) package.



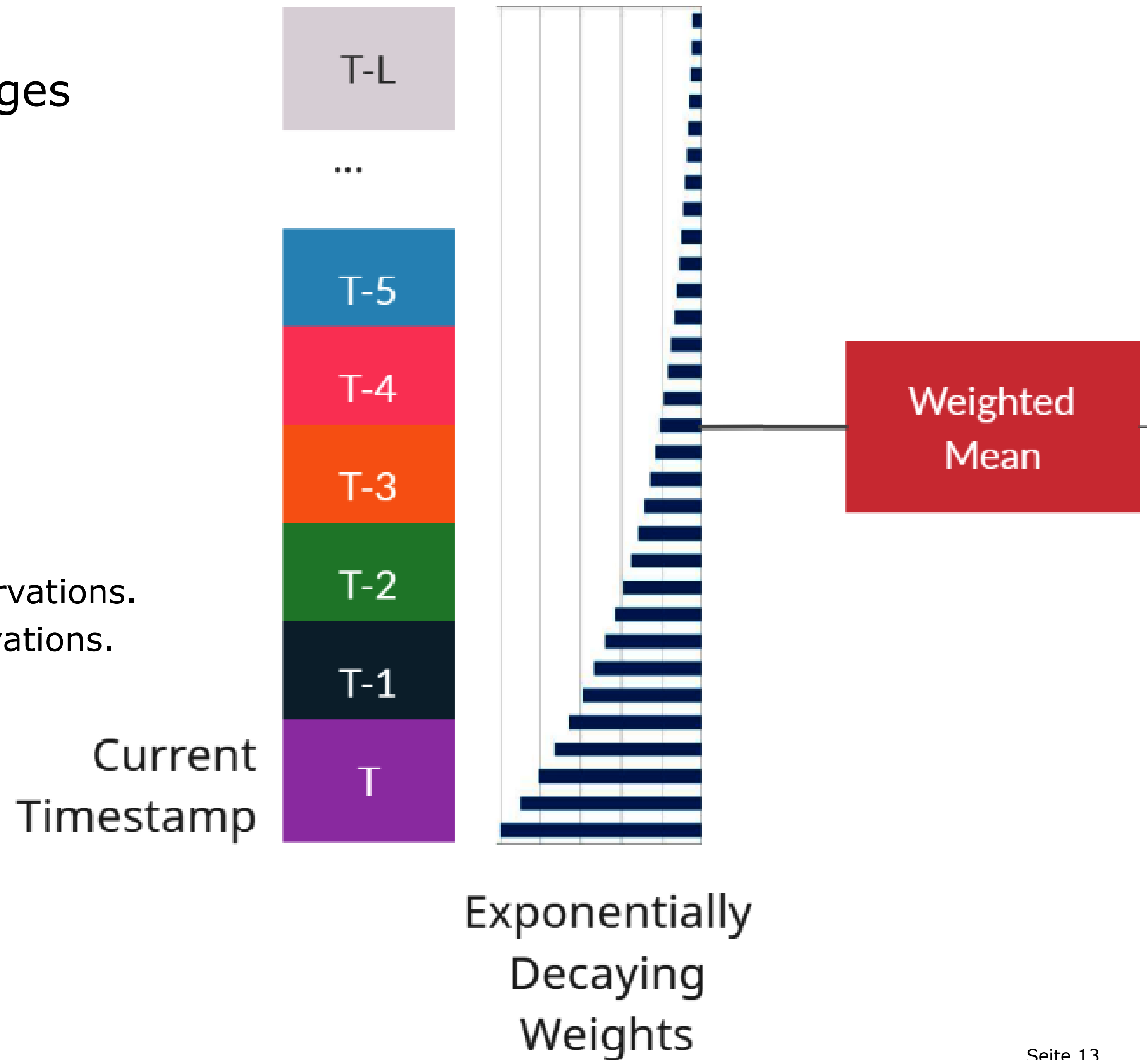
Exponentially weighted moving averages

Recall simple exponential smoothing,
for $\alpha \in [0,1]$ the weight for x_{t-k} is $w_k = \alpha(1 - \alpha)^k$

Consider different values for α

- Large α assigns more weight to **recent** observations.
- Small α assigns more weight to **older** observations.

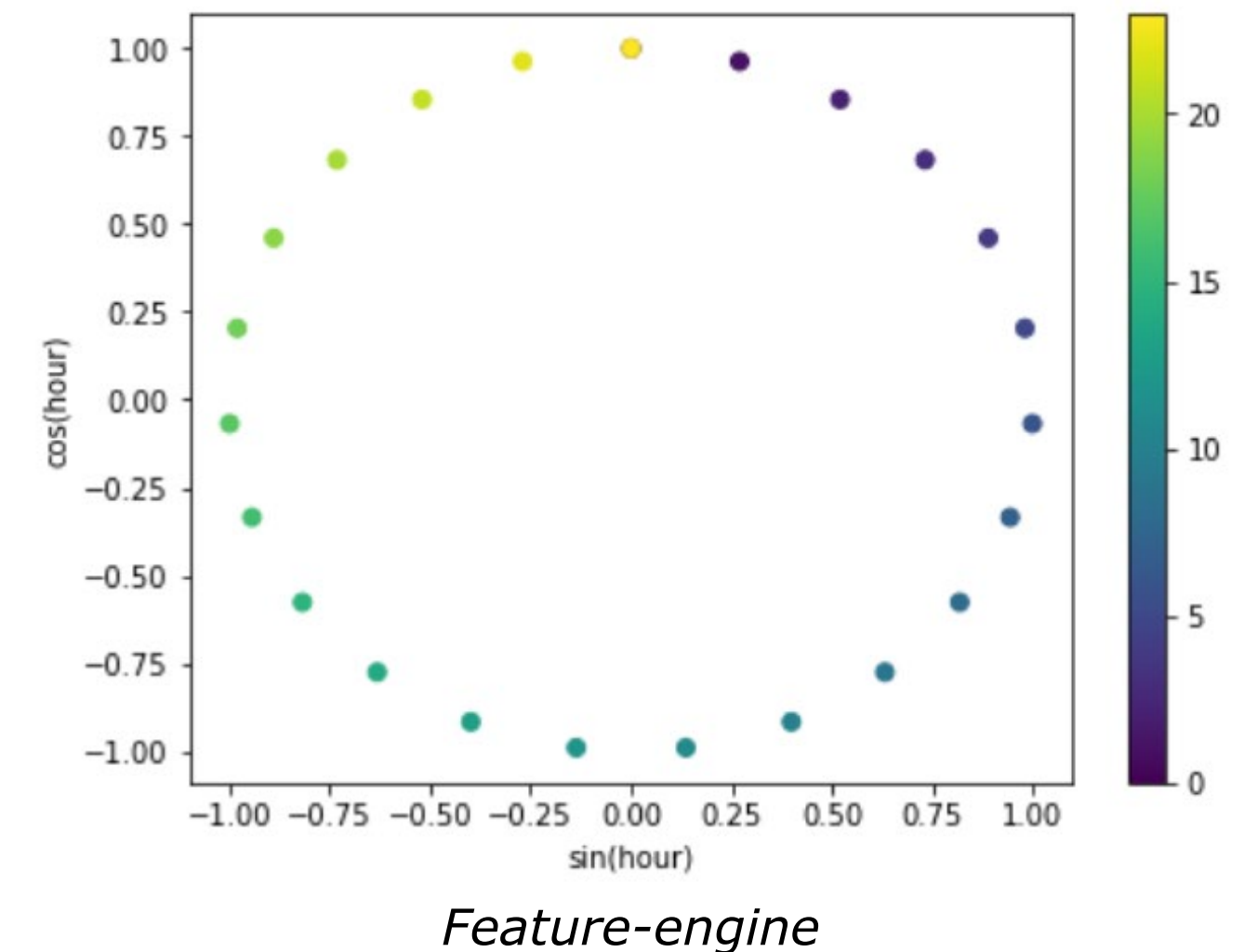
`pandas.DataFrame.ewm`



Temporal features

Calendar features: month, quarter, day, week, hour, ...

- Consider **cyclical encoding**: $x_{1,t} = \sin(x_t \cdot 2\pi / \max(x_t))$, $x_{2,t} = \cos(x_t \cdot 2\pi / \max(x_t))$.



Time elapsed: monotonically increasing feature, giving a sense of the passage of time.

Encode **special events** (e.g., holidays) with dummy variables.

- Spike variable: equal to one for the event of interest and zero elsewhere.
- Step variables: equal to one during the duration of the event (can be permanent e.g., change of trend).
- Beware of the dummy variable trap

Fourier terms for a seasonal period P :

Check `sktime` [FourierFeatures](#).

- $x_{1,t} = \sin(2\pi t \cdot 1/P)$, $x_{2,t} = \cos(2\pi t \cdot 1/P)$, $x_{3,t} = \sin(2\pi t \cdot 2/P)$, $x_{4,t} = \cos(2\pi t \cdot 2/P)$, ...

For more, check *Fulcher, Ben D. "Feature-based time-series analysis.", Feat. eng. for ml and data analytics. 2018.*

Feature importance and selection

Two types of feature importance.

- **Global**: importance over the whole dataset e.g., permutation, leave-one-out.
- **Local**: importance for a single data sample e.g., LIME, SHAP.

Filter methods: statistical methods independent of the model.

- Correlation analysis, statistical tests, mutual information, variance thresholding.
- Efficient but may not capture feature dependencies.

Wrapper methods: evaluate feature subsets by training a model and assessing its performance.

- Recursive feature elimination, forward/backward selection, genetic algorithms.
- Computationally expensive, model-specific, but can lead to better feature sets.

Embedded methods: feature selection integrated in model training process.

- Lasso/Ridge regression, decision trees, bagging/boosting.
- Efficient, model-dependent, requires tuning.

Data preparation – Cleaning



Identification and correction of errors.

- Mistypes, corruption, duplicates, inconsistencies...
- Verify data types (int, float, categorical, etc.)
- Formatting/structure harmonization
- Ensure **consistent** time intervals
- **Granularity** of observations: up/down-sampling

Identification and correction of anomalies & **outliers**.

- Using expert knowledge or statistical analysis: check data ranges, validate plausibility of correlations.

Drop/fill **missing** values (e.g. forward/backward, mean, interpolation), see scikit-learn [imputation of missing values](#).

- Why is the data missing? Is it missing at **random or is there a pattern?**
- How much data is missing, and could imputing it introduce **bias** or distort the analysis?
- Would **removing** the missing data be more appropriate in the context?

Document non-trivial cleaning operations & use dataset versioning e.g., DVC.

Data preparation – Transformations

Adapt data to match ML algorithm assumptions

Encoding variables

- Discretization: numeric (int, floats) to bins: KBinsDiscretizer
- Categorical to numeric: LabelEncoder
- Categorical to ordinal (with rank ordering): OrdinalEncoder
- Categorical to binary vector: OneHotEncoder

Transforming features

- Scaling features: MinMaxScaler, RobustScaler, StandardScaler
- Normalizing data samples to unit norm: Normalizer
- Log transform for skewed data: FunctionTransformer
- Make data more Gaussian: power_transform

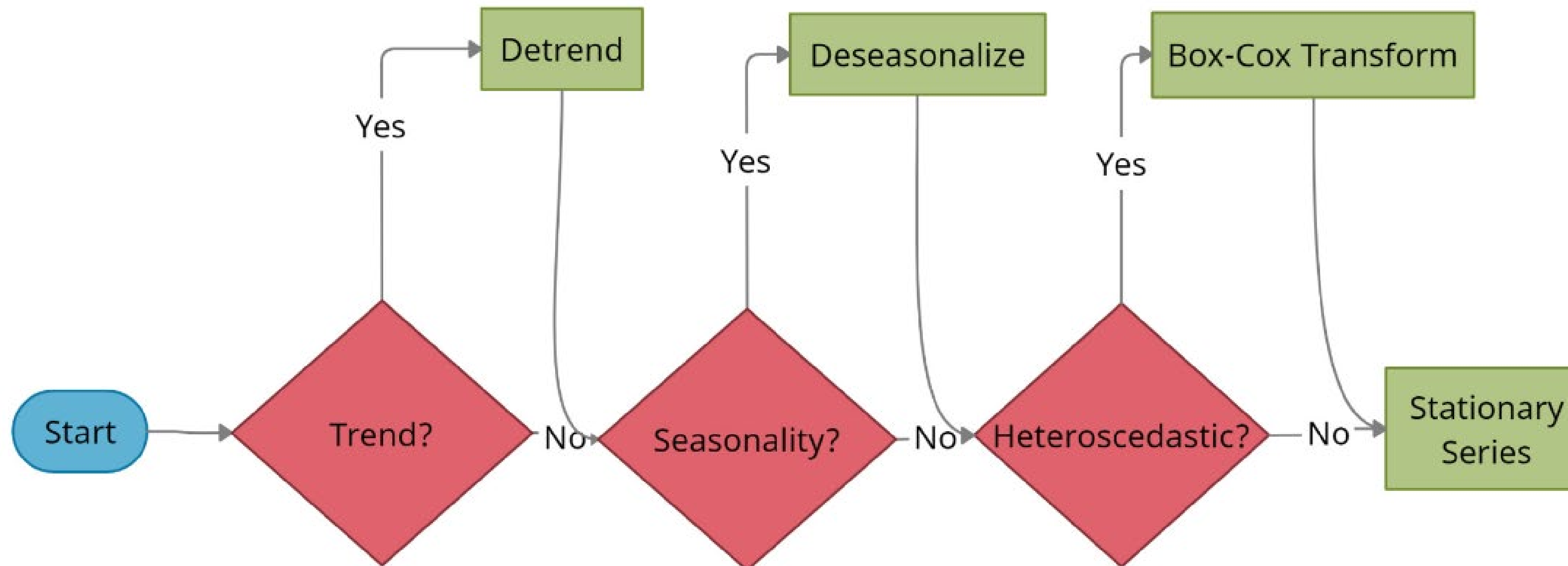
See scikit-learn [preprocessing data](#) and [dataset transformations](#)



Stationarity

Weak stationarity: the **statistical properties** of the time series remain **constant over time**.

Engineering **temporal features can be sufficient** to handle simple forms of non-stationarity. In complex case (or if the model assumes stationarity), making the TS stationary may be necessary.



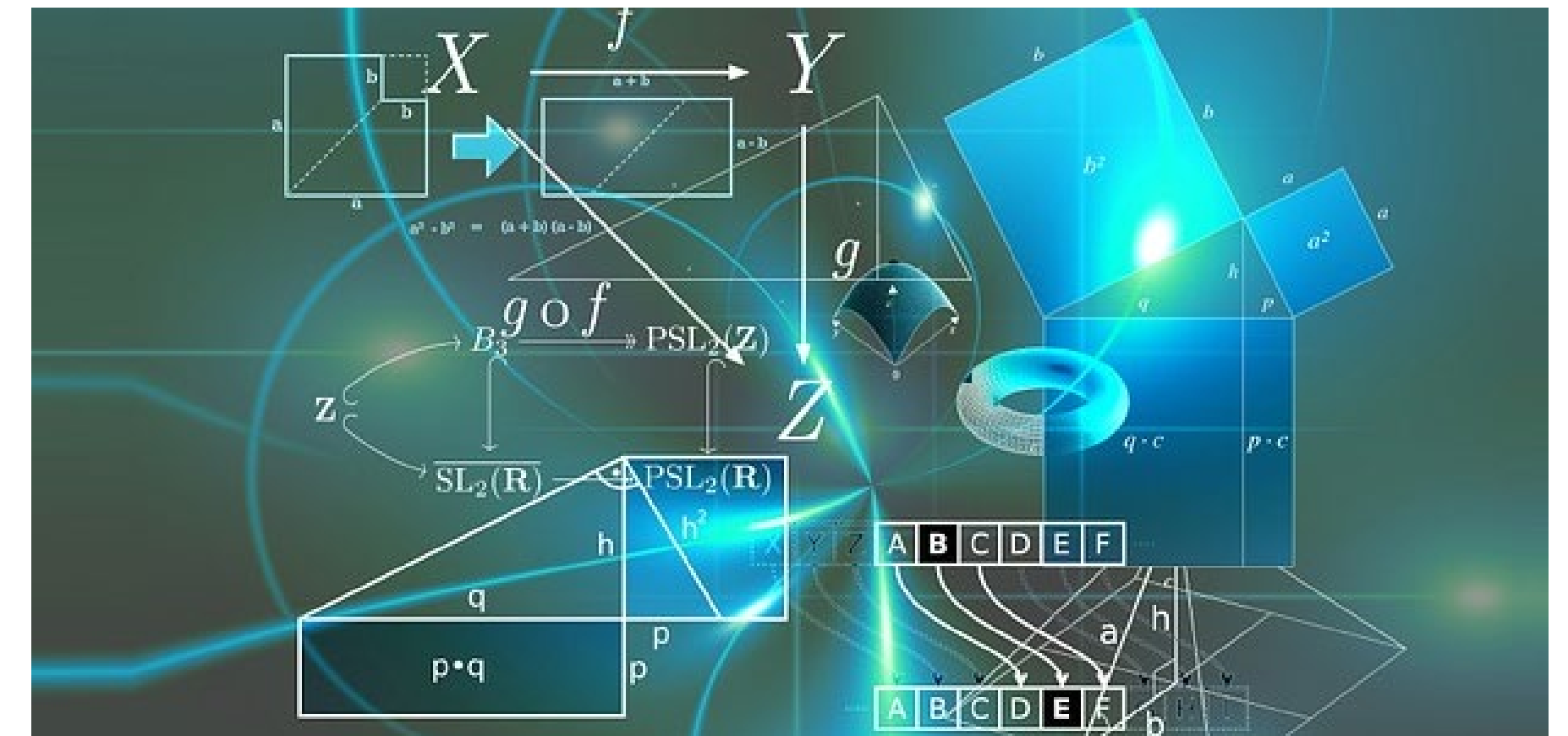
Tour of ML models

The machine learning algorithm zoo is huge

- Instance-based
- Regression
- Decision trees
- Artificial neural networks
- Etc.

Develop intuition/understanding of important algorithms

- Understand assumptions
- Hyperparameters selection
- Debug issues
- Error analysis



K-nearest neighbors

Select distance measure to compare data samples.

- Euclidean distance
- Cosine distance
- Manhattan distance

Select number of neighbors K.

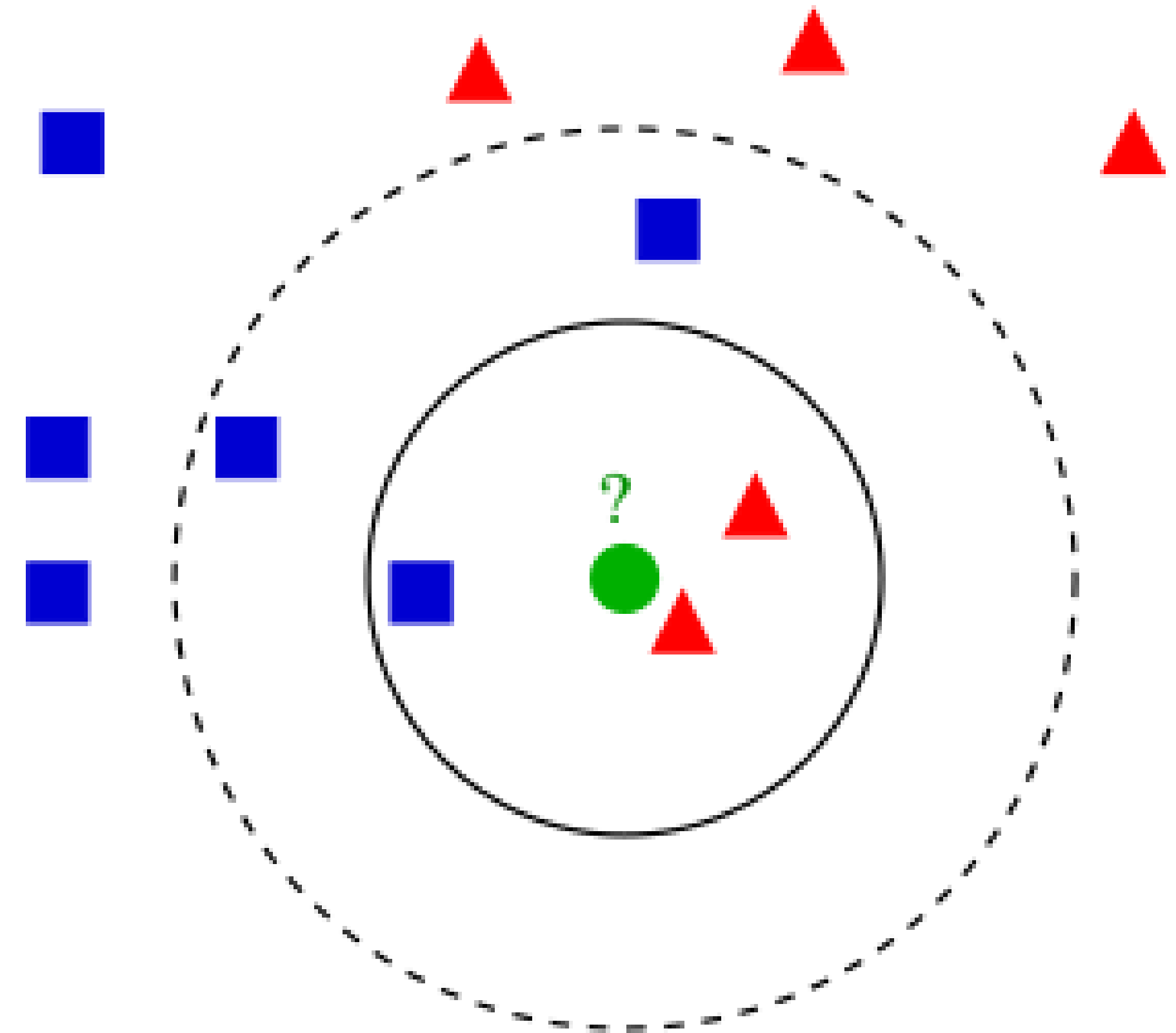
Classification: majority vote amongst neighbors.

Regression: (weighted) mean of neighbors.

`sklearn.neighbors.NeighborsClassifier`

`sklearn.neighbors.KNeighborsRegressor`

`sklearn.metrics.pairwise.distance_metrics`



Source: AnAj, Wikimedia Commons

Linear regression

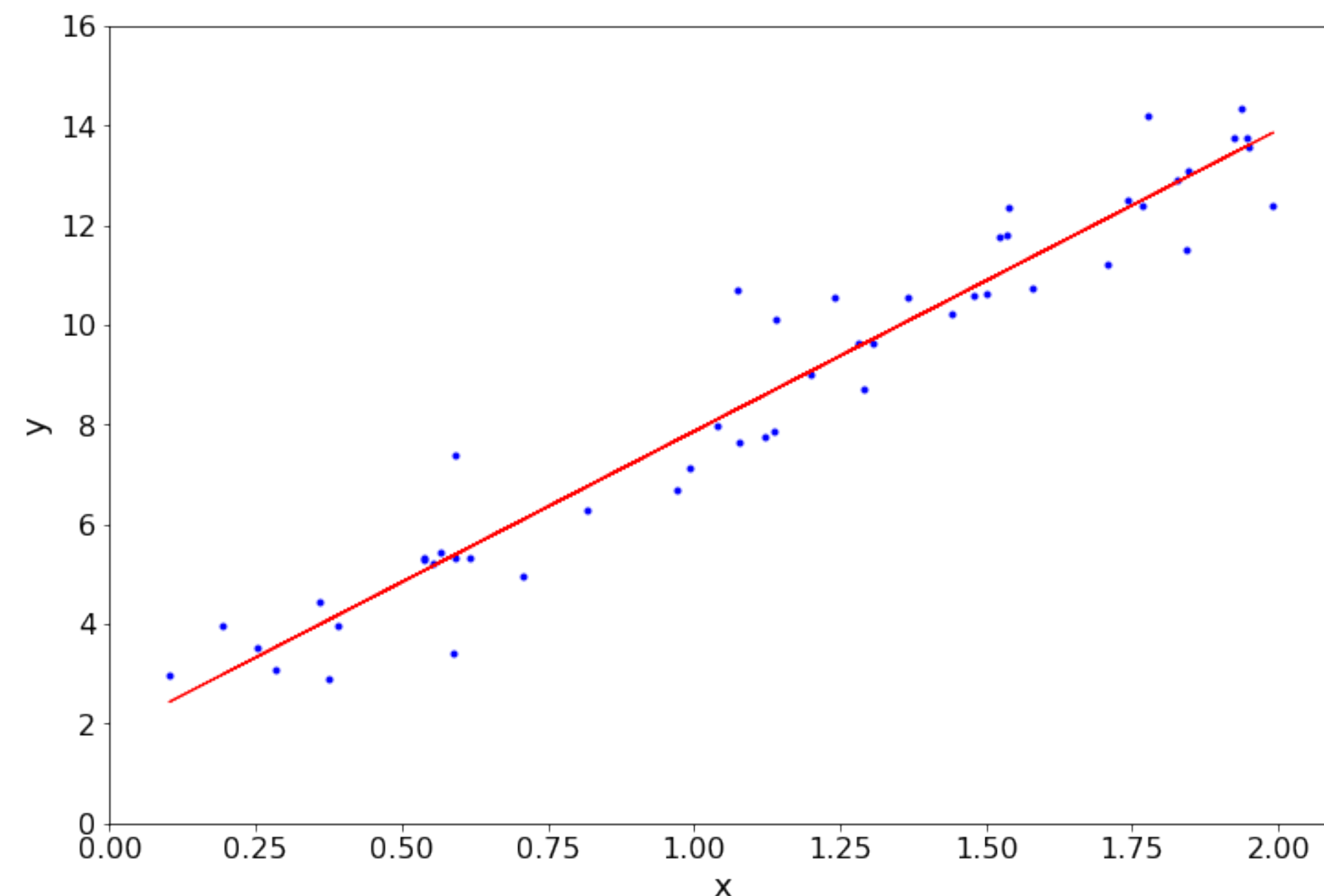
$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

with $\hat{y}$ the model prediction, θ_i the model parameters and x_i the data features

Performance measure based on e.g., the root-mean-square error: $RMSE = \sqrt{\frac{\sum_{i=1}^n (\hat{y}_i - y_i)^2}{n}}$

`sklearn.linear_model.LinearRegression`

`sklearn.metrics.mean_squared_error`



Polynomial regression

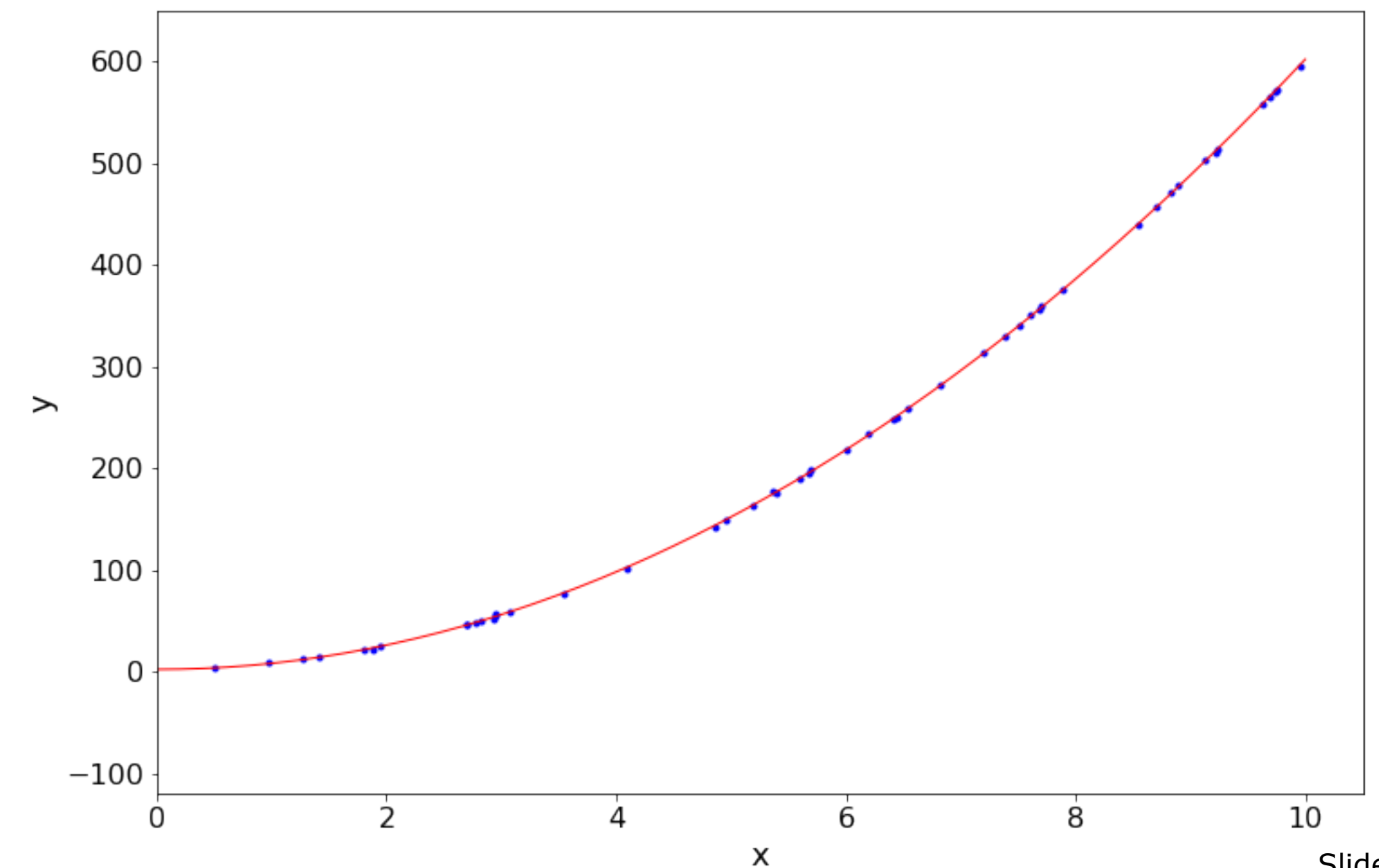
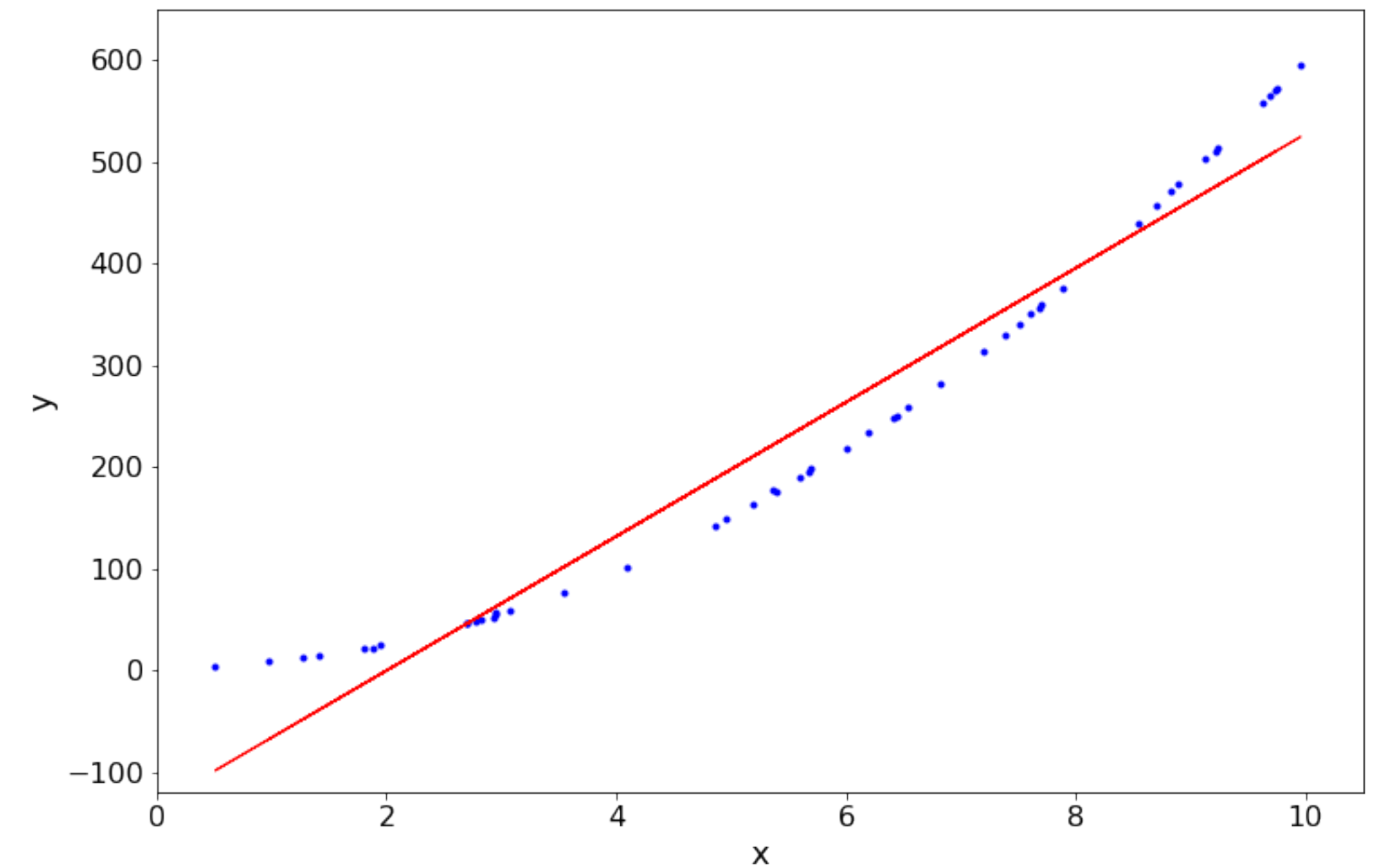
What if the data is non-linear?

Extend feature set with polynomial features.

For two features x_1 and x_2 , the degree-2 polynomial feature set is $\{x_1, x_2, x_1x_2, x_1^2, x_2^2\}$.

Apply linear regression on polynomial feature set.

`sklearn.preprocessing.PolynomialFeatures`



Regularization

Constrain model complexity to favor simpler models that are better at generalizing.

Polynomial regression: reduce polynomial degree.

Ridge regression: enforce small model parameters.

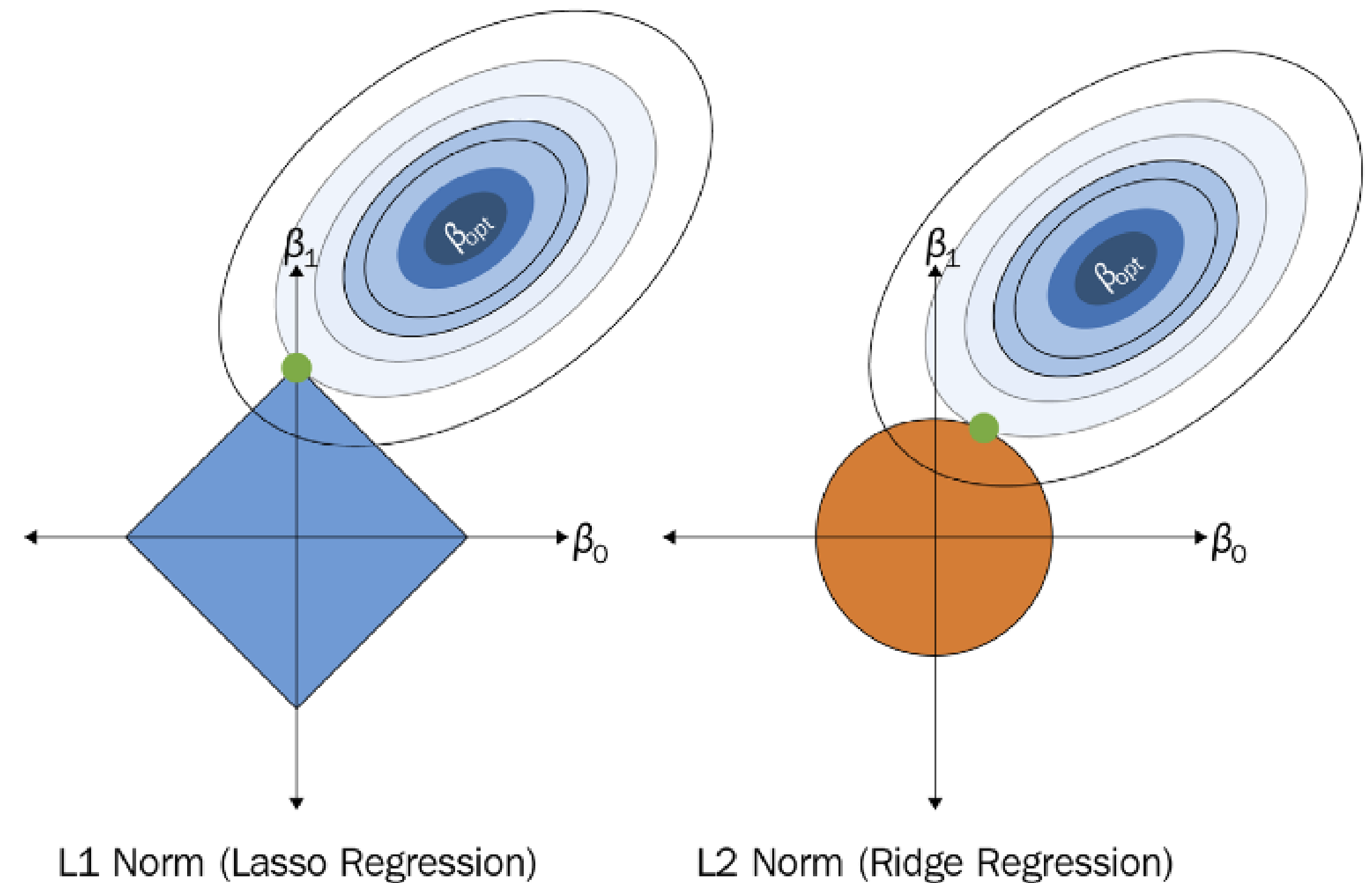
- Reduce model variance but increase bias.

Lasso regression: enforce sparse model parameters.

- Perform feature selection.

Elastic Net: mix of Ridge and Lasso regression.

See scikit-learn [linear models](#).



Joseph, Manu, and Jeffrey Tackes. *Modern Time Series Forecasting with Python*.

Logistic regression – classification

Standard regression where the output is passed to the sigmoid function $S(x) = \frac{1}{1 + e^x}$

- Model predicts a probability
- Produce binary classification based on a threshold

Performance measure based on e.g., binary cross-entropy

$$H = \sum_{i=1}^n y_i \log(p(\hat{y}_i)) + (1 - y_i) \log(1 - p(\hat{y}_i))$$

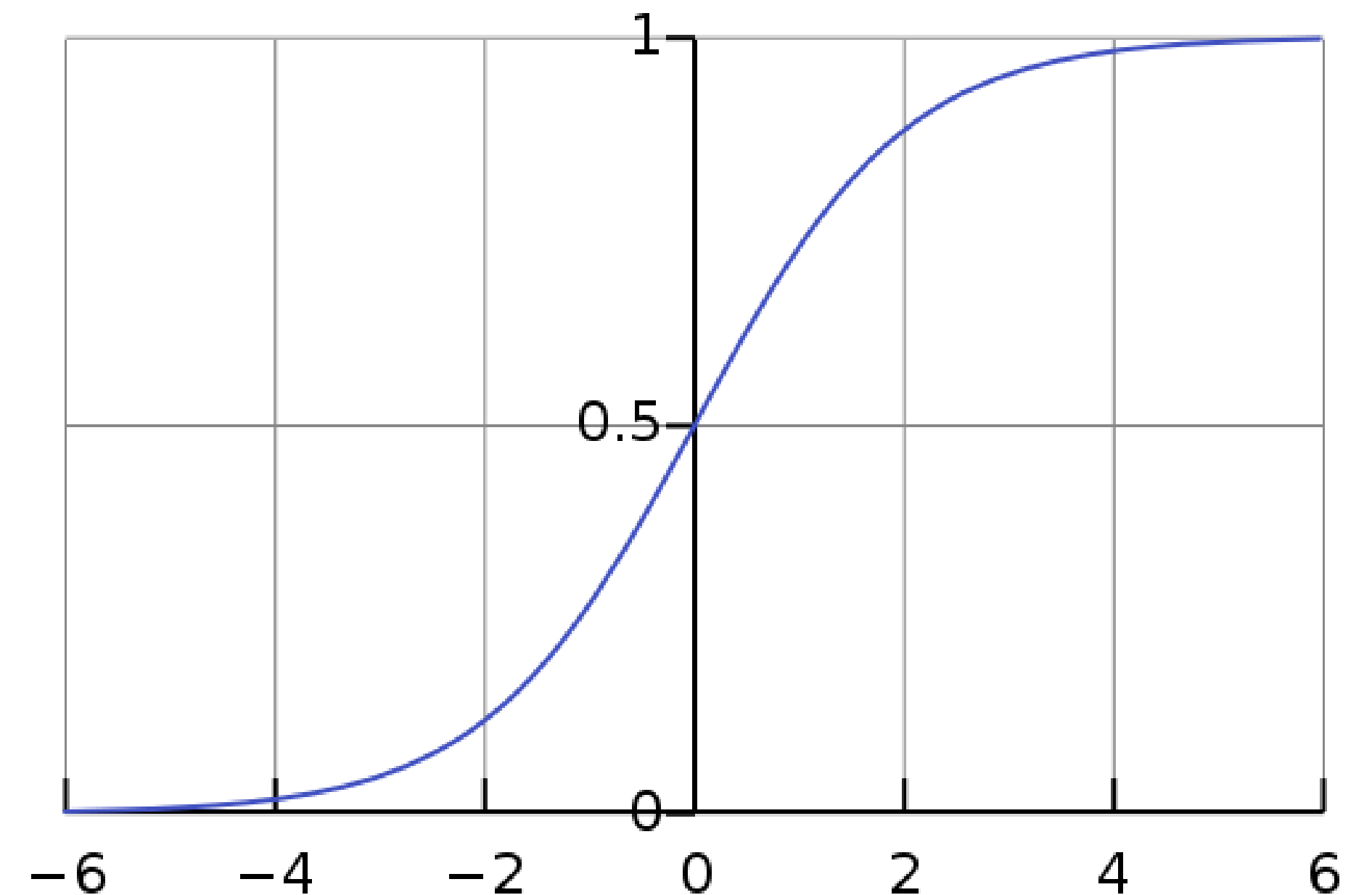
Multiclass classification: one-vs-all, one-vs-one, etc.

`sklearn.linear_model.LogisticRegression`

`sklearn.metrics.log_loss`

See scikit-learn [multiclass and multioutput algorithms](#)

Source: Qef, Wikimedia Commons



Support vector machines

The same dataset can be separated differently even by the same model due to randomness in the training process.
The data samples may be uncertain.

SVM finds separating decision boundary with largest margin.

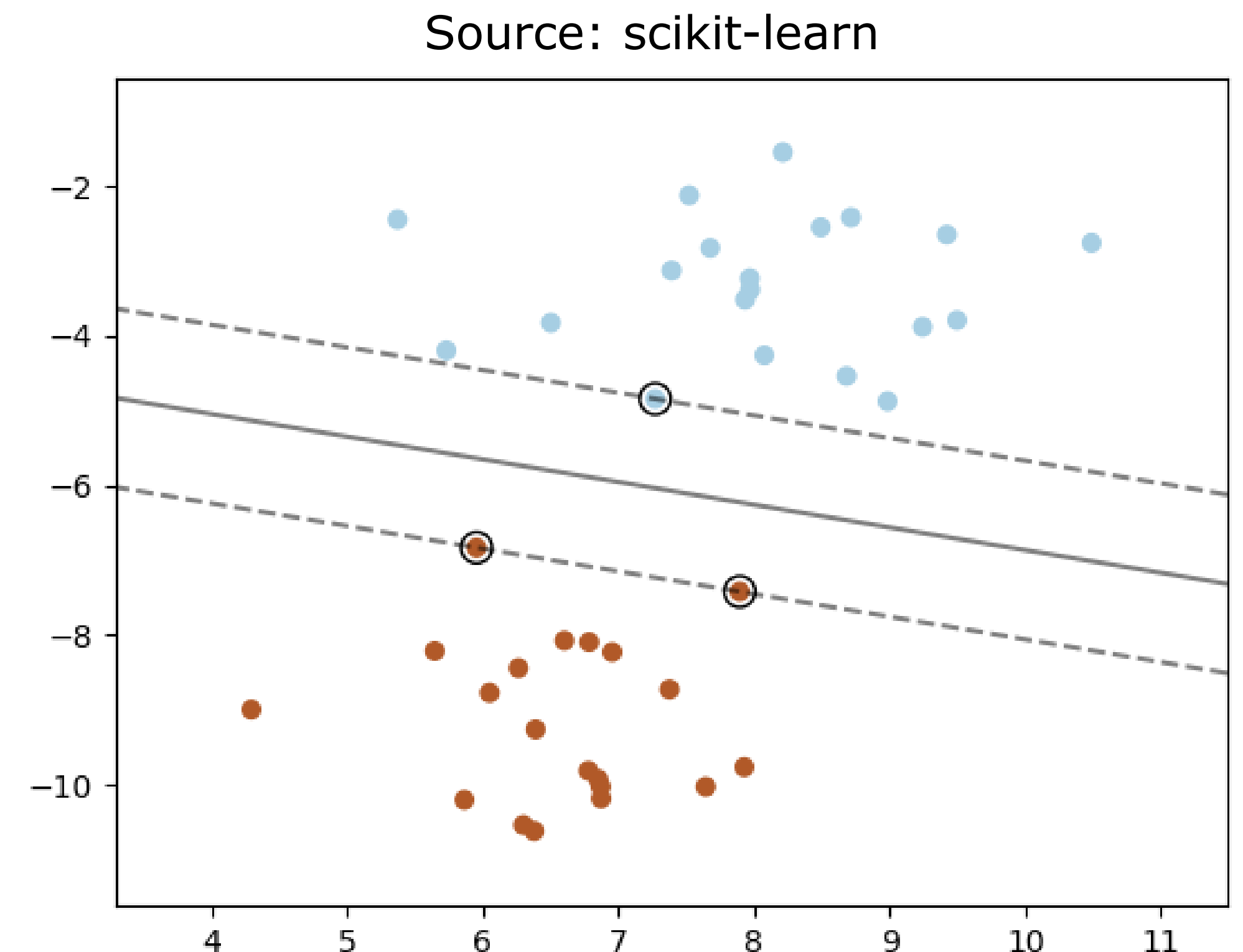
- Hard margin: weak against outliers
- Soft margin: more robust against outliers

Use the kernel trick for non-linear data.

- Polynomial kernel
- Gaussian RBF kernel

`sklearn.svm.LinearSVC`

`sklearn.svm.SVC`



Support vector machines – regression

Fit as many data samples as possible, while limiting samples outside margins.

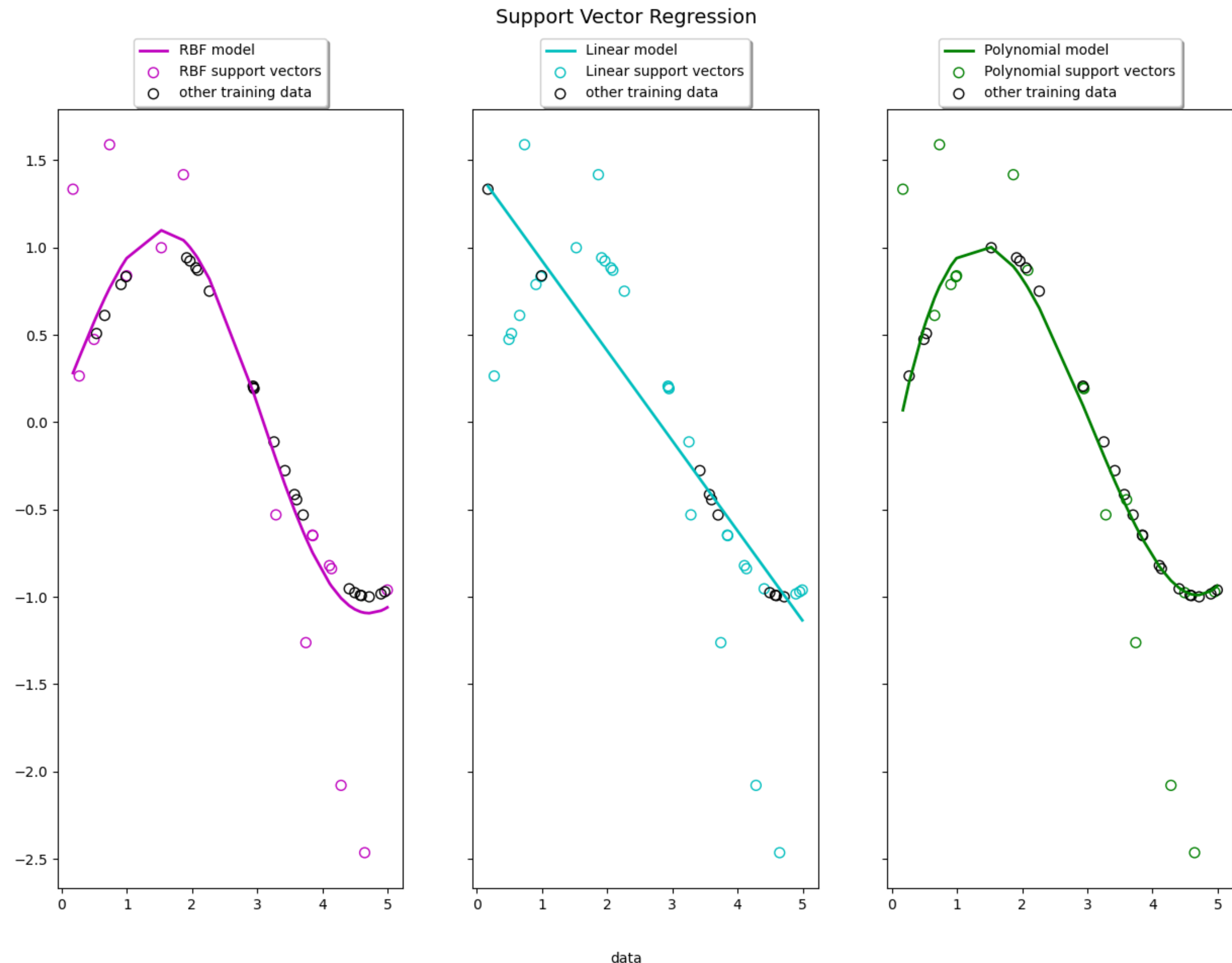
Margin width controlled with hyperparameters ϵ .

Data samples within margin do not affect model.

`sklearn.svm.LinearSVR`

`sklearn.svm.SVR`

See [scikit-learn support vector machines](#)



Source: scikit-learn

Decision trees

Recursive procedure: split on feature and threshold producing the “purest” subsets

- Information gain with gini/entropy for classification
- Variance reduction/mean squared error for regression

Regularization

- Max depth
- Min samples in leaves

Pros

- No data preparation required
- Can handle both numeric and categorical data

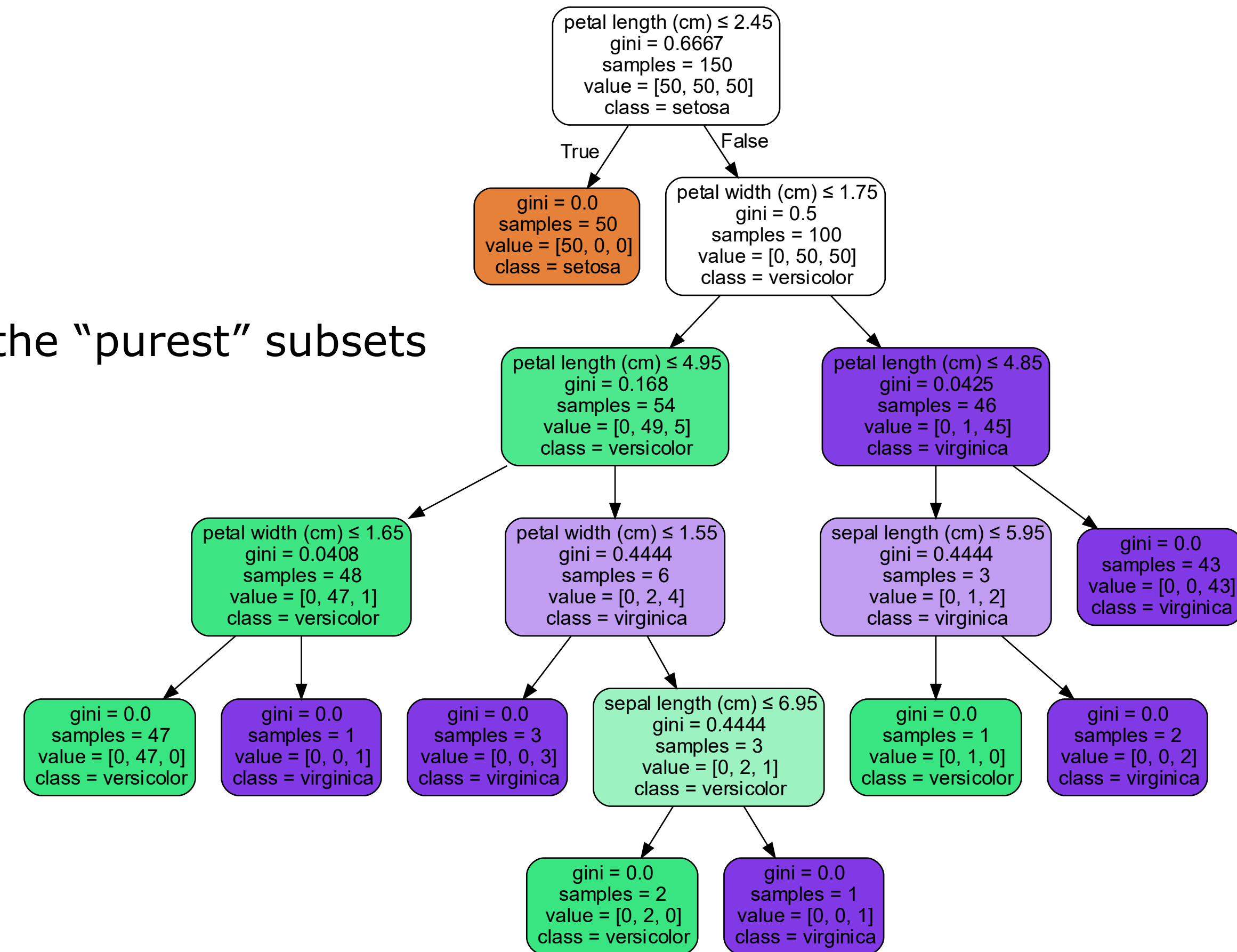
Cons

- Usually less accurate than other models and generalize poorly (overfit the data)
- Sensitive to data quality: small change results in different trees

`sklearn.tree.DecisionTreeClassifier`

`sklearn.tree.DecisionTreeRegressor`

Source: scikit-learn



See scikit-learn [decision trees](#)

Ensemble learning

Get strong predictor from the aggregation of weak predictors

Bagging: weak predictors trained on different random subsets of the training set

- Random forest: collection of decision trees trained from variations of the training set
 - Sampling of data samples with replacement
 - Sampling of data features without replacement

Boosting: train predictors sequentially, each correcting predecessor

- Adaboost: increase weights of misclassified samples
- GradientBoosting: fit new predictors on residual errors

Stacking: meta-predictor trained on weak predictors predictions

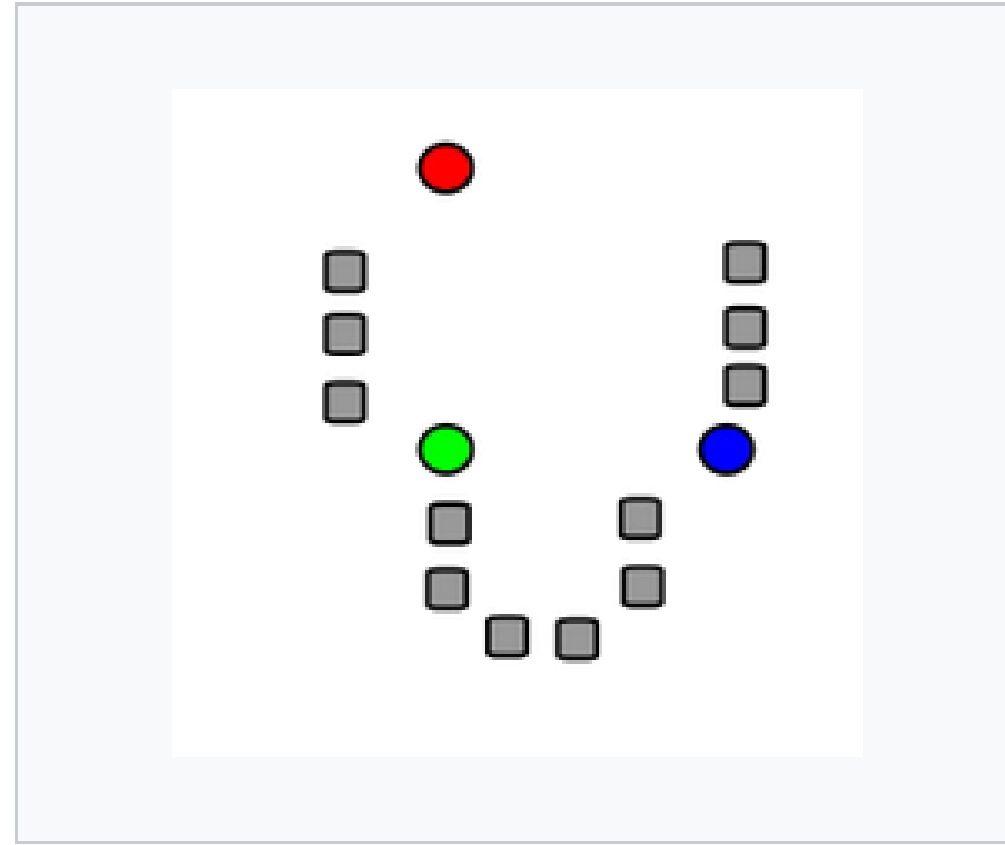
- Keep hold-out set to train meta-predictor



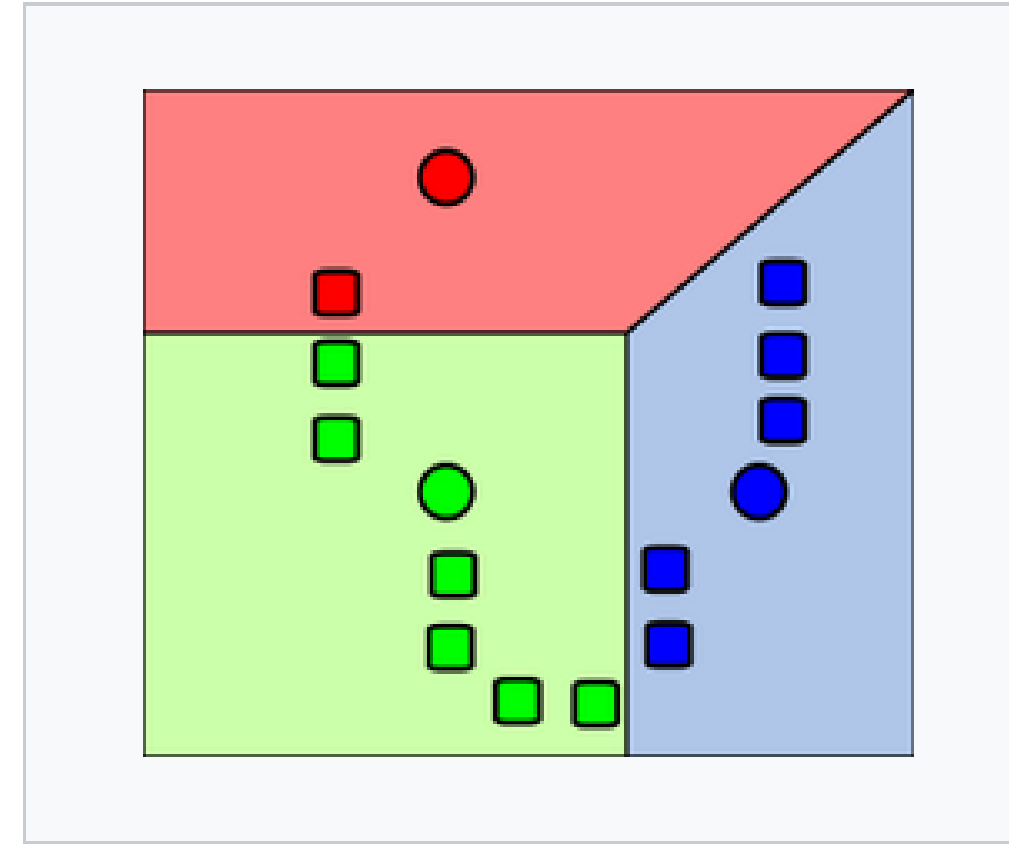
See scikit-learn [ensemble methods](#)

K-Means algorithm

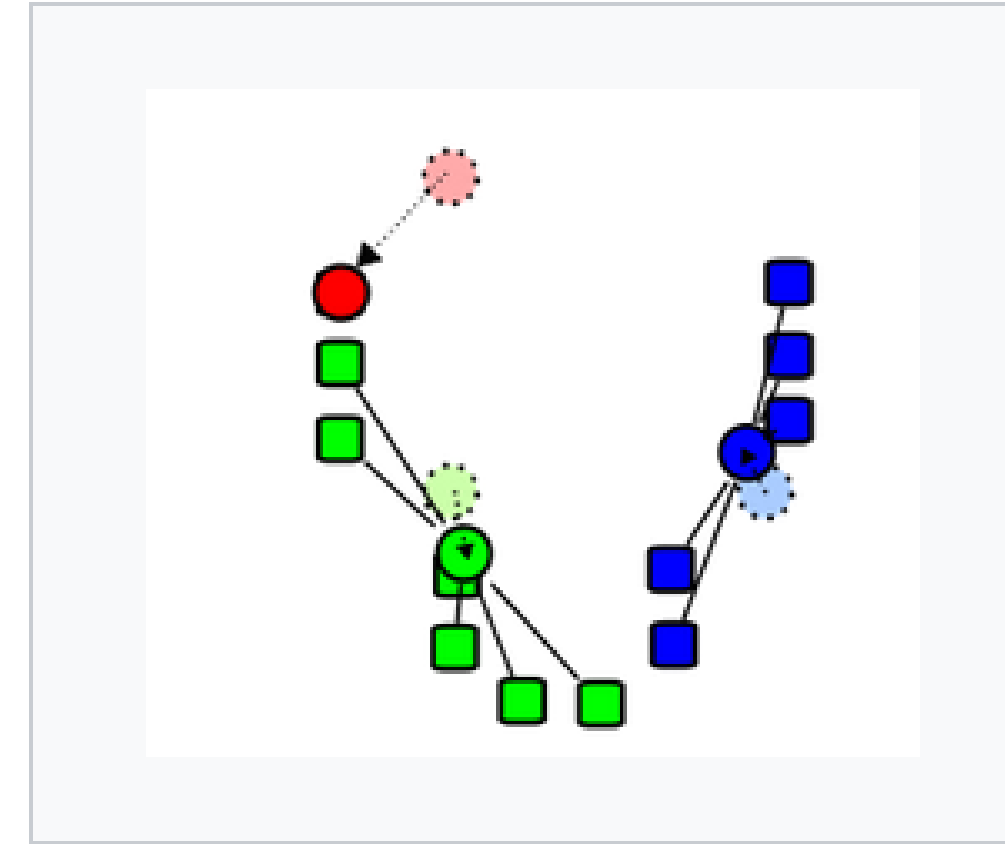
Source: Weston.pace, Wikimedia Commons



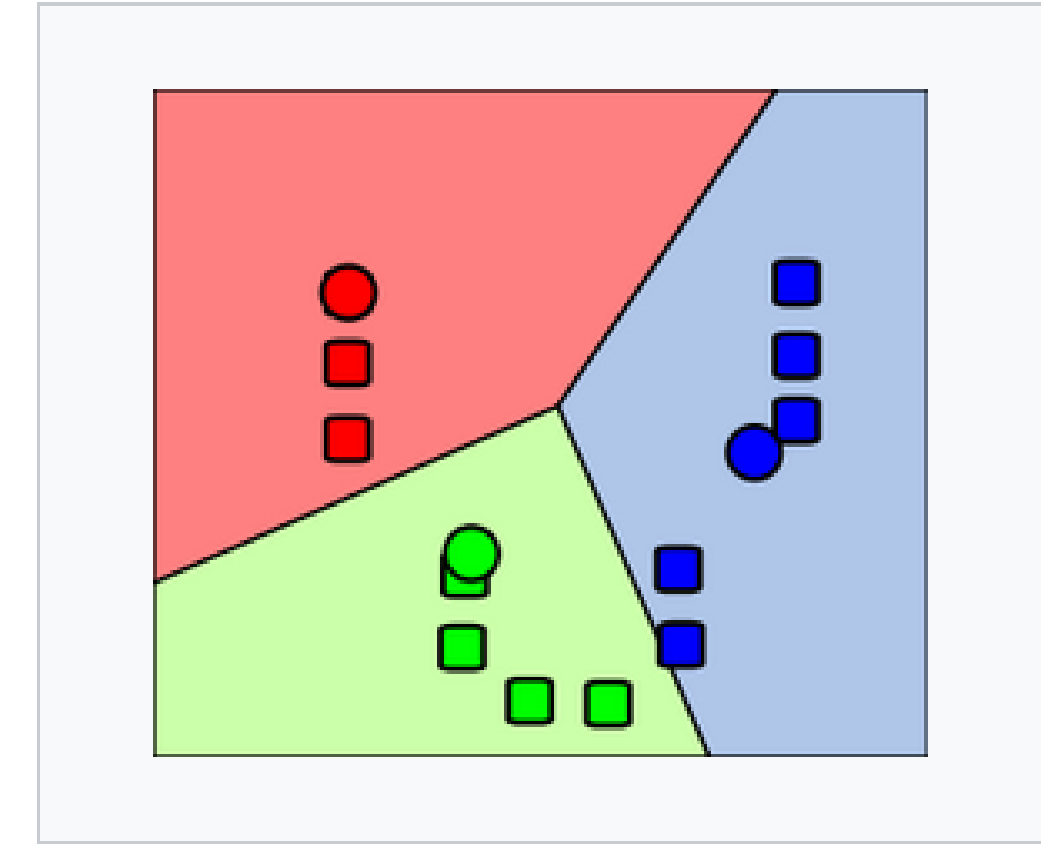
1. k initial "means" (in this case $k=3$) are randomly generated within the data.



2. k clusters are created by associating every observation with the nearest mean.



3. The centroid of each of the k clusters becomes the new mean.



4. Steps 2 and 3 are repeated until convergence has been reached.

Algorithm may converge to local optimum depending on

- Random initialization
- Selected number of centers K
- Selected distance measure
- Sensitive to outliers

`sklearn.cluster.KMeans`

See [scikit-learn clustering](#)

Modern ML baselines for time series

Gradient boosting: XGBoost, LightGBM, CatBoost.

- Often SOTA in industry forecasting with limited data.

Distance-based: Dynamic Time Warping + kNN

- Use elastic similarity measures. Does not require feature engineering.
- Excellent baselines for time-series classification.

Shapelet-based

- Learn discriminative subsequences ("shapelets").
- Good balance between interpretability and accuracy.

Dictionary/Bag-of-Patterns: BOSS, cBOSS, WEASEL

- Convert series into symbolic patterns; classify via histogram features.
- Robust to noise and warping.

Random Convolutional Kernel: ROCKET, MiniROCKET, MultiROCKET

- Thousands of random convolution features + linear classifier.
- Extremely fast and highly accurate for classification.

Evaluation workflow

1. **Split dataset** into **training**, **validation**, and **test** sets (splitting method depends on the learning task).
 - Validation set is required either when model fitting involves hyperparameters tuning or when model selection is based on performance metrics.
 - Multiple versions of training and validation sets can be considered with cross-validation.
2. **Train** candidate models on training set.
 - Tune hyperparameters using the validation set.
3. **Select model** based on model fit, complexity and performance on the validation set.
 - Information criteria, residual/error analysis, uncertainty, performance metrics
4. **Train selected model** on training + validation sets then **evaluate performance** on test set.
 - Metrics provide an indication of how well the model will **generalize to new data**.

Challenges – Limited training data

In practice, only a limited amount of training data is available $\tilde{X} \subseteq X$.

$$\theta^* = \max_{\theta} \sum_{\forall x \in \tilde{X}} P(m_{\theta}(x), t(x))$$

The model parameters θ^* may be optimal on $\tilde{X}$ but suboptimal on $X \setminus \tilde{X}$.

Thus, **$\tilde{X}$ should be as representative as possible of X with the same underlying data distribution.**

How much data is needed?

This depends on complexity of task and chosen algorithm.

- Number of input features (curse of dimensionality).
- Number of target classes (for classification problems).
- Number of model parameters.

Analyze published studies tackling similar problems.

Can be answered only through empirical investigation.

- Plot model performance vs dataset size.

In general, the more data, the better: Halevy, Alon, Peter Norvig, and Fernando Pereira. "The unreasonable effectiveness of data." IEEE intelligent systems 24.2 (2009): 8-12.



Challenges – Bias sources

Data collection

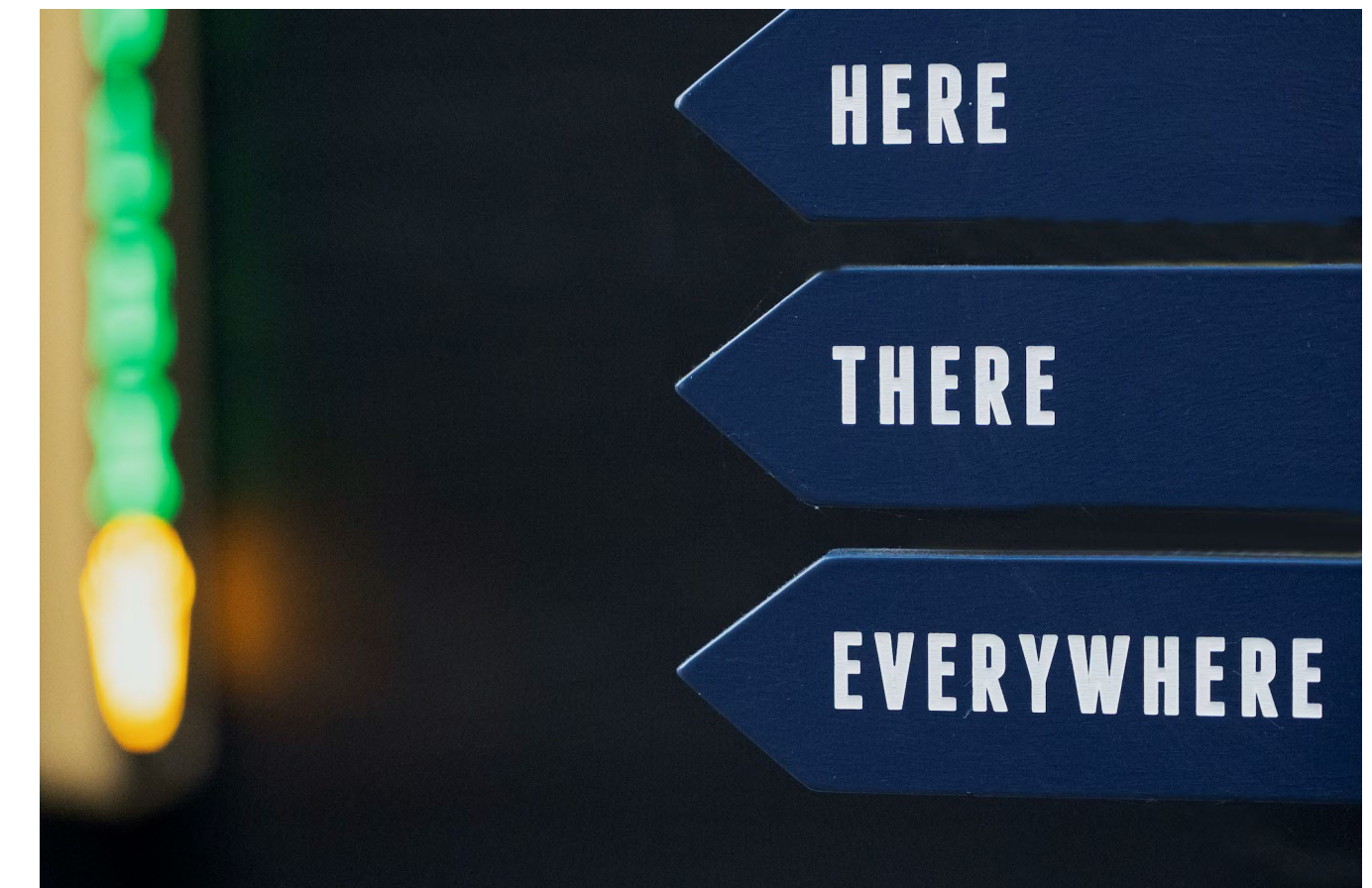
- Capturing process e.g., healthcare access, patient consent
- Inclusion criteria
- Geographic: data collected from limited geographic regions
- Population demographics: age, gender, ...
- Collection centers: specialized centers with different disease prevalence

Annotation

- Variability in the expertise level.
- Intra and inter-observer variations

Evolution over time

- Seasonal bias
- Data drift



Data probability distribution must be/remain consistent between training and inference.

Causality

- In causal setting, i.e. X causes Y, covariate shift and concept shift are considered.
- In anti-causal setting, i.e. Y causes X, label shift and manifestation shift are considered.
- Covariate and label shifts are indistinguishable when causality is unknown.

Differences in data distribution can introduce bias and degrade model performance.

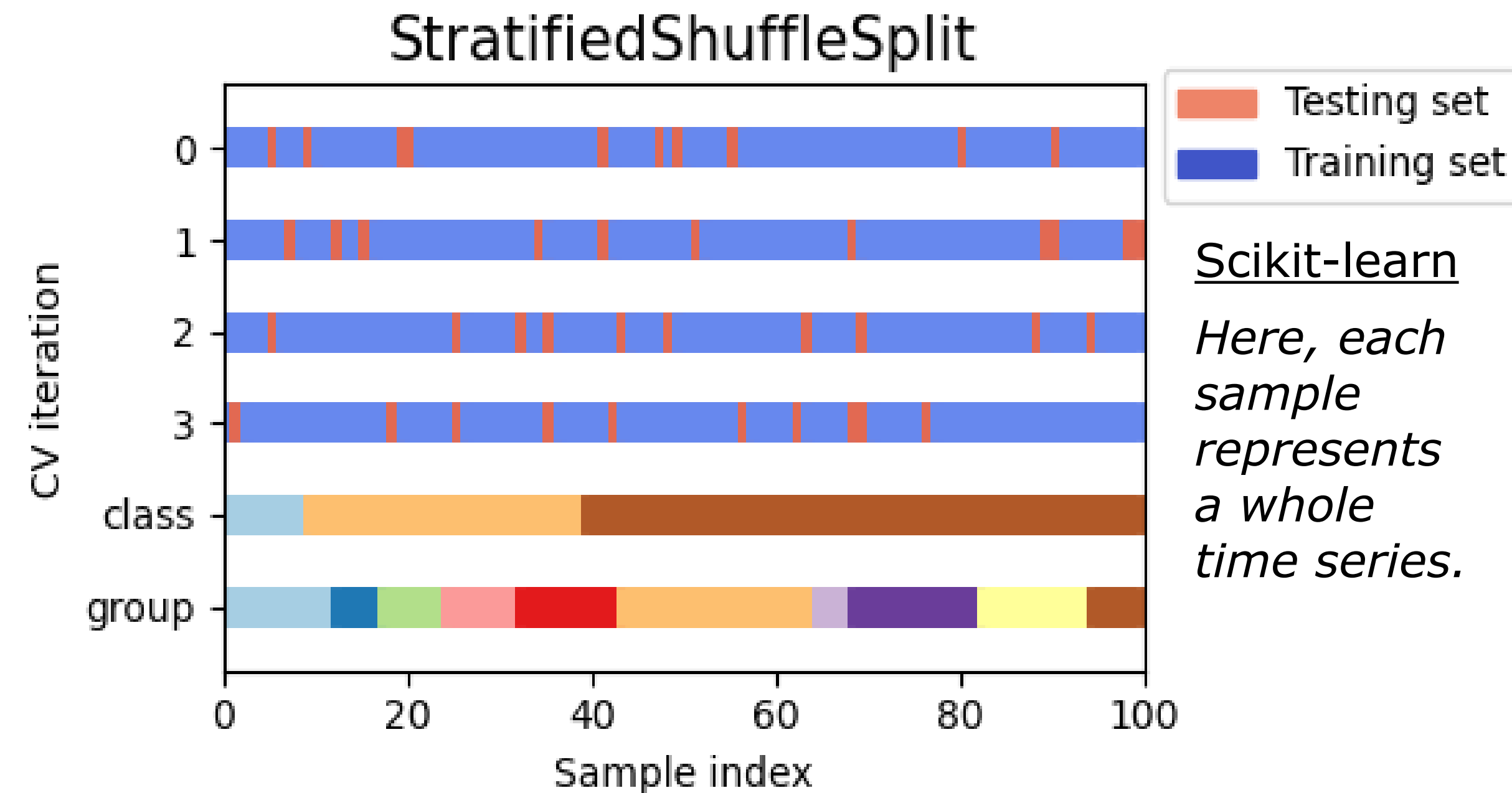
Type of shift	Data $P(X)$	Targets $P(Y)$	Concept $P(Y X)$	Manifestation $P(X Y)$
Covariate shift	Change	Change	Same	Same
Concept shift	Same	Change	Change	Same
Label shift	Change	Change	Same	Same
Manifestation shift	Change	Same	Same	Change

$$P(Y, X) = P(Y|X)P(X) = P(X|Y)P(Y) = P(X, Y)$$

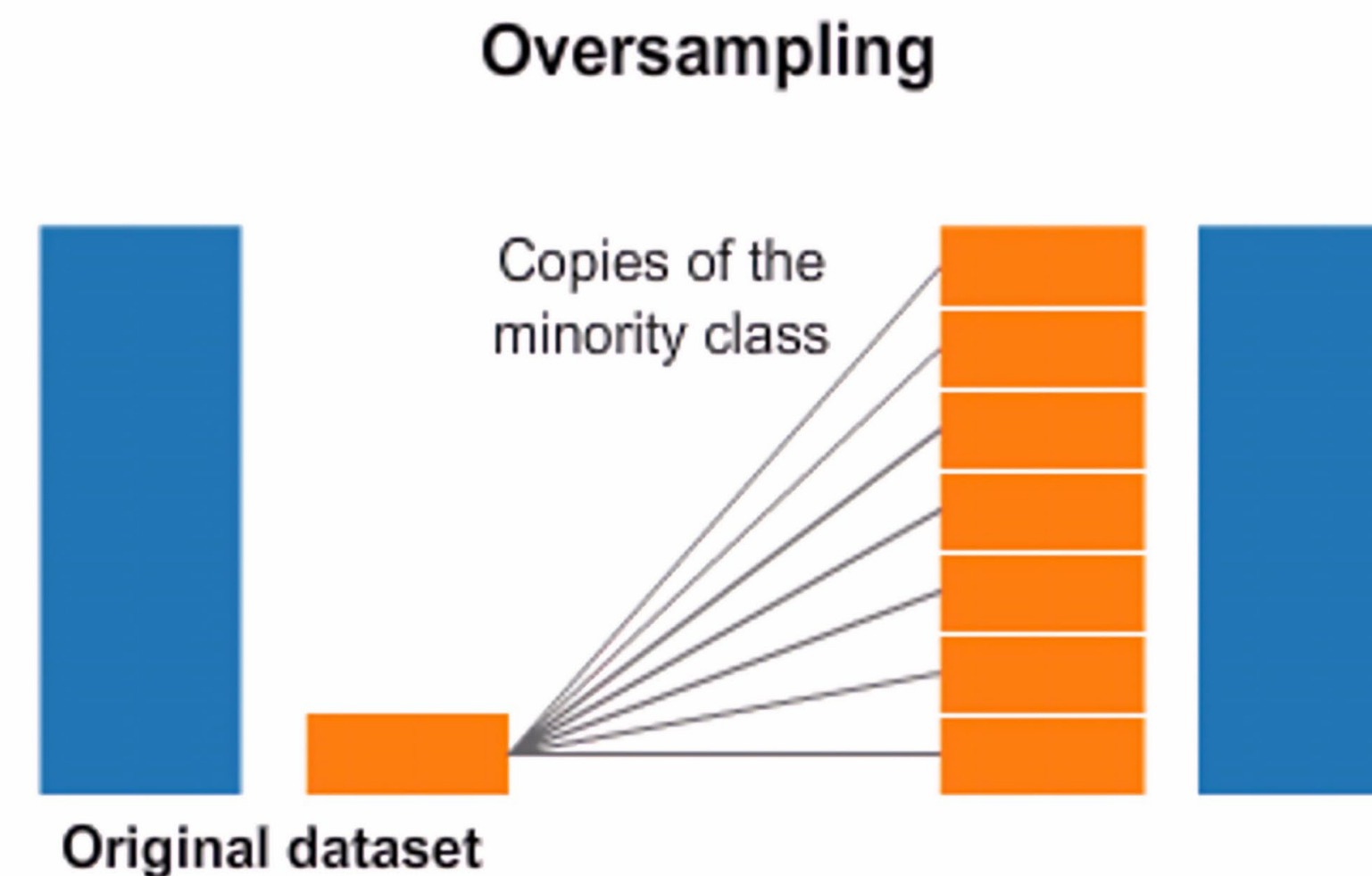
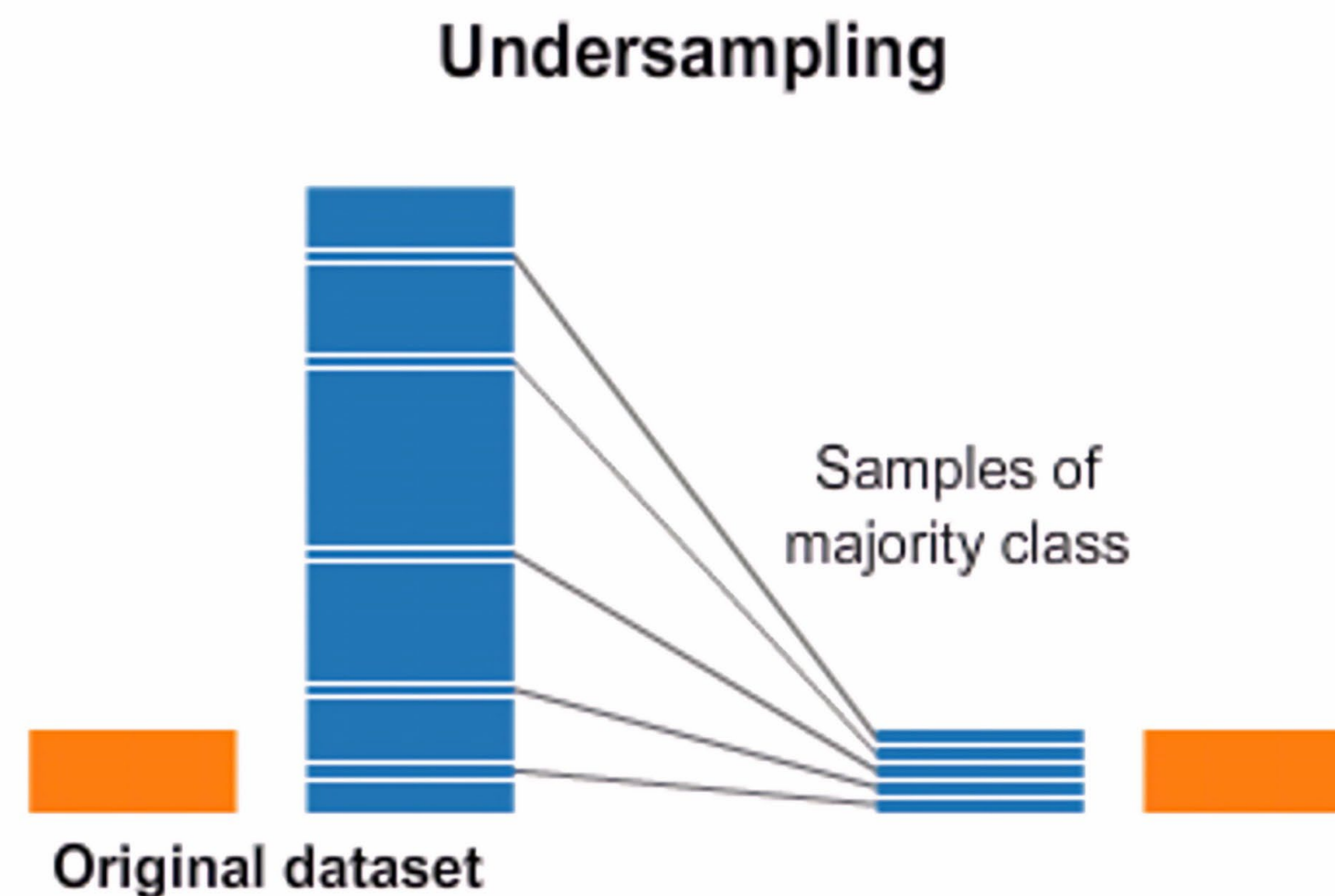
Challenges – Imbalanced dataset

Imbalance in **classification** tasks

- Use **stratified** splits to preserve target distribution.
- Merge **minority classes**.



Imbalance-aware metrics: **balanced accuracy**, precision/recall, F1-score, area under precision/recall curve.



Liu et al. 2019

Limitations of classical ML for time series

ML models require extensive **manual** feature engineering.

They assume **fixed-length** feature windows → cannot naturally handle variable-length sequences.

Poor at capturing **long-range dependencies** or complex nonlinear temporal dynamics.

Struggles with **multivariate interactions** and high-frequency signals without heavy preprocessing.

No native support for autoregressive **sequence generation**.

Limited probabilistic forecasting capabilities, **uncertainty requires ad hoc wrappers**.

Representation learning is not native to ML → requires hand-tuning and limits scalability.

Deep learning learns hierarchical temporal representations directly from **raw data**.

Exercise

Review Kaggle tutorials:

- Machine learning
- Intermediate machine learning
- Time series

Model real-world time series.

- Engineer temporal features.
- Train time series models.
- Make data stationary and retrain models.
- Compare with classical time series models such as ARIMA and ETS.

For the last lecture, review *Dempster et al. "ROCKET: exceptionally fast and accurate time series classification using random convolutional kernels." Data Mining and Knowledge Discovery 34.5 (2020): 1454-1495.*